

Rules and Strategies for Transforming Functional and Logic Programs

ALBERTO PETTOROSSÌ

University of Rome Tor Vergata

and

MAURIZIO PROIETTI

IASI CNR

We present an overview of the program transformation methodology, focusing our attention on the so-called “rules + strategies” approach in the case of functional and logic programs. The paper is intended to offer an introduction to the subject. The various techniques we present are illustrated via simple examples.

Categories and Subject Descriptors: D.1.1 [**Programming Techniques**]: Applicative (Functional) Programming; D.1.2 [**Programming Techniques**]: Automatic Programming; D.1.6 [**Programming Techniques**]: Logic Programming; D.2.4 [**Software Engineering**]: Program Verification—*correctness proofs*; D.2.5 [**Software Engineering**]: Testing and Debugging—*symbolic execution*; D.2.10 [**Software Engineering**]: Design—*methodologies*; D.3.4 [**Programming Languages**]: Processors—*optimization*; F.2.0 [**Analysis of Algorithms and Problem Complexity**]: General; F.3.1 [**Logics and Meaning of Programs**]: Specifying and Verifying and Reasoning about Programs; F.3.2 [**Logics and Meanings of Programs**]: Semantics of Programming Languages; I.2.2 [**Artificial Intelligence**]: Automatic Programming—*program transformation*; I.2.3 [**Artificial Intelligence**]: Deduction and Theorem Proving—*logic programming*.

General Terms: Algorithms, Design, Documentation, Theory, Verification

Additional Key Words and Phrases: Program derivation, program improvement, transformation rules, transformation strategies

1. INTRODUCTION

The program transformation approach to the development of programs was advocated by Burstall and Darlington [1977], although the basic ideas had been

This work has been partially supported by the Progetto Finalizzato Sistemi Informatici e Calcolo Parallelo of the CNR, Italy, under grant n. 89.00026.69, MURST 40%, and Esprit Compulog II. A preliminary version of this paper has been published in *Formal Program Development. Lecture Notes in Computer Science 755*, B. Möller, H. Partsch, and S. Schuman, Eds. Springer Verlag, 1993, pp. 263–304. Authors’ addresses: A. Pettorossi, Electronics Department, University of Rome Tor Vergata, Via della Ricerca Scientifica, I-00133, Rome, Italy; email: <adp@iasi.rm.cnr.it> ; M. Proietti, IASI CNR, Viale Manzoni 30, I-00185, Rome, Italy; email: <proietti@iasi.rm.cnr.it>.

Permission to make digital/hard copy of part or all of this work for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication, and its date appear, and notice is given that copying is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee.

© 1996 ACM 0360-0300/96/0300-0360 \$03.50

CONTENTS

1. INTRODUCTION
 2. TRANSFORMATION RULES AND STRATEGIES FOR FUNCTIONAL PROGRAMS
 - 2.1 Transformation Rules for Functional Programs
 - 2.2 Transformation Strategies for Functional Programs
 - 2.3 Examples of Transformations of Functional Programs
 3. TRANSFORMATION RULES AND STRATEGIES FOR LOGIC PROGRAMS
 - 3.1 Transformation Rules for Logic Programs
 - 3.2 Transformation Strategies for Logic Programs
 - 3.3 Examples of Transformations of Logic Programs
 4. OTHER TECHNIQUES FOR TRANSFORMING FUNCTIONAL AND LOGIC PROGRAMS
 - 4.1 Schemata Approach
 - 4.2 Partial Evaluation
 - 4.3 Finite Differencing
 5. RELATED TECHNIQUES FOR PROGRAM DEVELOPMENT AND FINAL DISCUSSION
 - ACKNOWLEDGMENTS
 - REFERENCES
-

presented in previous papers by the same authors [Darlington 1972; Burstall and Darlington 1975].

In that approach the task of writing a correct and efficient program is realized in phases: the first phase consists of writing an initial, maybe inefficient, program whose correctness can be easily shown, and the second phase, possibly divided into various subphases, consists in transforming the initial program in order to get a new program that is more efficient. This methodology may avoid the difficulty of designing the invariant assertions of loops which are often quite intricate, especially for very efficient algorithms.

The experience gained by the scientific community during the past two decades shows that the methodology of program transformation is very valuable and attractive, in particular for the task of programming “in the small,” that is, when writing single modules of large software systems.

Program transformation also has the advantage of being adaptable to various programming paradigms. In this article we focus our attention on the functional and logic cases.

The basic idea of the program transformation approach is depicted in Figure 1. From the initial program P_0 , which is the given specification, we want to obtain a final program P_n with the same semantic value, that is, $Sem(P_0) = Sem(P_n)$ for some given semantic function Sem . This is often done in various steps, by constructing a sequence $\langle P_0, \dots, P_n \rangle$ of programs such that $Sem(P_i) = Sem(P_{i+1})$ for $0 \leq i < n$.

Sometimes, if the initial program is nondeterministic (i.e., it may produce a set of answers for any given input), we may allow transformation steps that are *sound*, but not *complete*, in the sense that $Sem(P_n)(v) \subseteq Sem(P_0)(v)$, for any input value v .

A given cost function C which measures the space or time complexity of the execution of a program should satisfy the following inequation: $C(P_0) \geq C(P_n)$. We may allow ourselves to derive a program version, say P_i , for some $i > 0$, such that $C(P_0) < C(P_i)$, because subsequent transformations may lead to a program version whose cost is smaller than the one of P_0 . Unfortunately, there is no general theory of program transformations that deals with this situation in a satisfactory way.

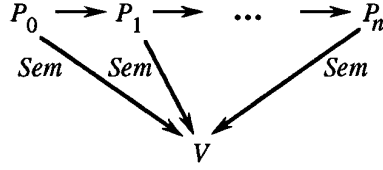


Figure 1 Program transformation idea: from program P_0 to program P_n preserving the semantic value V .

2. TRANSFORMATION RULES AND STRATEGIES FOR FUNCTIONAL PROGRAMS

In this section we use a programming language similar to the one presented in Burstall and Darlington [1977], in which programs are written as sets of recursive equations. Many examples are given in the following. The extension to the case of functional languages such as ML or Miranda is straightforward.

We need the following notions. The expression e_1 is an *instance* of the expression e_2 iff there exists a substitution θ so that $e_1 = e_2\theta$. For example, $f(x + 3)$ is an instance of $f(x + y)$ and the matching substitution θ is $\{y = 3\}$.

A *context expression* $C[\dots]$ is an expression C in which one or more occurrences of a given subexpression are “deleted.” For example, $f(x + (\dots + 1))$ is a context expression.

The semantics of a program is assumed to be the *least fixed point* of the corresponding set of recursive equations and thus, while transforming programs, we want to preserve the least fixed point semantics.

The equations of any given program are assumed to be: *mutually exclusive*, that is, two different left-hand sides do not have common instances; and *exhaustive*, that is, for any element v of the domain of any function, say f , defined by the recursive equations, there exists at least one left-hand side that matches $f(v)$.

2.1 Transformation Rules for Functional Programs

The rules for transforming programs expressed as recursive equations are the following ones:

(1) *Definition Rule*. It consists of adding to the current program P a set of mutually exclusive and exhaustive recursive equations, say $f(t_1) = e_1, \dots, f(t_n) = e_n$, where f is a function symbol not occurring elsewhere in P . Thus the left-hand side of any equation introduced by definition is not an instance of the left-hand side of any equation already existing in the current program.

(2) *Unfolding Rule*. It consists of replacing an occurrence of an instance of the left-hand side (l.h.s.) of an equation by the corresponding instance of its right-hand side (r.h.s.), thereby producing a new equation. For example, if we unfold the subexpression $f(n + 1)$ in $f(n + 2) = (n + 2) \times f(n + 1)$ using the equation $f(n + 1) = (n + 1) \times f(n)$, we get the new equation:

$$f(n + 2) = (n + 2) \times ((n + 1) \times f(n)).$$

(3) *Folding Rule*. It is the inverse of the unfolding rule. It consists of replacing an occurrence of an instance of the r.h.s. of an equation by the corresponding instance of its l.h.s., thereby producing a new equation.

(4) *Instantiation Rule*. It consists of the introduction of an instance of an already existing equation. For example, by instantiating n to $m + 1$ in the equation $f(n + 1) = (n + 1) \times f(n)$, we get, after simplification, $f(m + 2) = (m + 2) \times f(m + 1)$.

(5) *Where-Abstraction Rule*. We replace a recursive equation $f(\cdot \cdot \cdot) = C[e]$ by the new equation: $f(\cdot \cdot \cdot) = C[z]$ **where** $z = e$, provided that z is a variable that does not occur in $f(\cdot \cdot \cdot) = C[e]$. The use of the where-clause has the advantage that in the call-by-value mode of execution, the evaluation of the subexpression e is performed only once.

(6) *Algebraic Replacement Rule*. We derive a new equation by using equalities that hold when we interpret function variables as least fixed points of the corresponding recursive equations, and we interpret basic function symbols, such as $+$, $\times$, and the like, as operators in a given algebra. For instance, the equation $f(n + 1) = f(n) \times (n + 1)$ can be derived from the equation $f(n + 1) = (n + 1) \times f(n)$, if we interpret $\times$ as the usual multiplication operation on natural numbers, by using the equality $f(n) \times (n + 1) = (n + 1) \times f(n)$.

Given the following recursive equations that define the function *sum*:

$$\text{sum}([\])=0,$$

$$\text{sum}(a:l)=a+\text{sum}(l),$$

we have the equality, called *distributivity* of *sum* w.r.t. **if-then-else**:

$$\text{sum}(\text{if } x \text{ then } y \text{ else } z) = \text{if } x \text{ then } \text{sum}(y) \text{ else } \text{sum}(z),$$

which can be shown by computational induction [Manna 1974].

In this article, we assume that an equality $\text{expr}_1 = \text{expr}_2$ holds w.r.t. a given program P iff, for all values of the free variables in the expressions expr_1 and expr_2 , either the evaluations of expr_1 and expr_2 using the program P are both nonterminating, or they are both terminating and yield the same value.

If the equality $\text{expr}_1 = \text{expr}_2$ holds w.r.t. P we also say that expr_1 is *equivalent* to expr_2 w.r.t. P , and when no confusion arises, we feel free to omit the reference to program P .

New programs are obtained from old ones by deriving new equations through the use of the definition, unfolding, folding, instantiation, where-abstraction, and algebraic replacement rules and then selecting a set of mutually exclusive and exhaustive equations. It can be shown that by using the preceding six transformation rules, partial correctness is preserved, that is, if the derived program terminates (by using a fixed point computation rule) then it computes the value computed by the initial program [Kott 1978; Courcelle 1990; Sands 1994].

However, during program transformation we usually need to preserve total correctness, that is, the initial program terminates iff the derived program terminates, and in the case of termination, both programs compute the same result. Thus, after applying the transformation rules aforementioned, we should also check that the derived program terminates for all input values for which the initial program terminates.

2.2 Transformation Strategies for Functional Programs

When performing program transformations we may end up with a final program that is equal to the initial one (recall that the folding rule is the inverse of the unfolding rule). Thus, during the transformation process, we need *strategies* that guide the application of the transformation rules and may allow us to derive programs with improved performance.

One of the most relevant steps in applying a transformation strategy is the introduction of new functions, often called in the literature *eureka functions*. In the

early days of the transformation methodology, those functions were generated by clever insights (hence their name) and not by strategies.

Unfortunately, there is no general theory of strategies which ensure that the derived programs are more efficient than the initial ones. However, partial results are available for some classes of programs.

We now present some of the transformation strategies that have been proposed in the literature. The basic ideas on which those strategies are based, are present, in various forms, in a large number of papers on program transformation, program synthesis, and automated theorem proving. Our references to the literature are not exhaustive, and they only indicate some relevant papers where the reader may also find material for further study. We see the strategies in action in the following examples.

(i) *Composition Strategy* [Burstall and Darlington 1977; Darlington 1981; Feather 1982; Paige and Koenig 1982; Bauer et al. 1987; Partsch 1990]. If the subexpression $f(g(x))$ occurs in an expression e which is the r.h.s. of a recursive equation in a given program,

—we introduce, by the definition rule, a new function h defined as follows:

$$h(x) = f(g(x)),$$

—we find recursive equations for $h(x)$ where the functions f and g do not occur, and

—by folding we replace $f(g(x))$ by $h(x)$ in e , and possibly elsewhere in the given program.

Variants of the composition strategy are the *Internal Specialization* technique [Scherlis 1981] and the *Deforestation* technique [Wadler 1988].

By using the composition strategy (or its variants) one may avoid the construction of intermediate data structures that are produced by g and used as inputs for f . This strategy often allows the derivation of programs with improved performance w.r.t. those obtained by lazy evaluation [Wadler 1985] if the intermediate data structures are infinite and the “next item” of the output of f can be produced by knowing only a finite portion of the output of g .

(ii) *Tupling Strategy* [Burstall and Darlington 1977; Pettorossi 1977; Darlington 1981; Feather 1982; Paige and Koenig 1982; Bauer et al. 1987; Partsch 1990]. Given a recursive equation of the form $f(x_1, \dots, x_n) = C[f_1(e_1), \dots, f_r(e_r)]$, where the expressions $e_1, \dots, e_r$ all share the free variable x ,

—by using the definition rule we introduce the function:

$$h(x, y_1, \dots, y_m) = \langle f_1(e_1), \dots, f_r(e_r) \rangle,$$

where $x, y_1, \dots, y_m$ are the free variables occurring in $e_1, \dots, e_r$;

—we find recursive equations for $h(x, y_1, \dots, y_m)$ without occurrences of the functions $f_1, \dots, f_r$; and

—by where-abstraction and folding we replace the given equation $f(x_1, \dots, x_n) = C[f_1(e_1), \dots, f_r(e_r)]$ by the new equation:

$$f(x_1, \dots, x_n) = C[u_1, \dots, u_r]$$

where

$$\langle u_1, \dots, u_r \rangle = h(x, y_1, \dots, y_m),$$

where $u_1, \dots, u_r$ are fresh variables. By using the algebraic replacement rule we also replace elsewhere in the program the occurrences of $f_i(e_i)$ by $\pi_i(h(x, y_1, \dots, y_m))$, where π_i denotes the i th projection function, for $i = 1, \dots, r$.

The use of the tupling strategy is very useful if the functions $f_1, \dots, f_r$ all have access to the same data structure x which is used by no other function. In this case the store used for x can be released as soon as the value of h has been computed. Often, by doing so, one may improve the time $\times$ space performance [Pettorossi 1984]. The tupling strategy is also very effective when several functions require the computation of the *same subexpression*, in which case we tuple together those functions.

By avoiding either multiple accesses to data structures or common subcomputations one often gets linear recursive programs (i.e., equations whose r.h.s. has at most one recursive call) from nonlinear recursive programs.

(iii) *Generalization Strategy*. There are three forms of this strategy, as we now indicate.

- (1) *Generalization from expressions to variables* [Boyer and Moore 1975; Wegbreit 1976; Burstall and Darlington 1977; Aubin 1979; Darlington 1981; Feather 1982; Bauer et al. 1987; Partsch 1990]. Given a recursive equation of the form $f(x_1, \dots, x_n) = e$, such that a subexpression s occurs in e ,

—we introduce the generalized recursive equation:

$$g(x, y_1, \dots, y_m) = e[x/s],$$

where x is a fresh variable, $e[x/s]$ denotes the expression e with x substituted for s , and $x, y_1, \dots, y_m$ are the free variables occurring in $e[x/s]$;

—we find recursive equations for $g(x, y_1, \dots, y_m)$; and

—we express the function $f(x_1, \dots, x_n)$ in terms of $g(x, y_1, \dots, y_m)$ by folding one or more instances of $e[x/s]$.

- (2) *Generalization from functions to functions by implicit definition* [Darlington 1981; Pettorossi 1984]. Given a recursive equation of the form: $f(x_1, \dots, x_n) = C[expr]$ in a program P ,

—we introduce the function g with arity k , such that there exist some expressions $e_1, \dots, e_k$, possibly with the variables $x_1, \dots, x_n$, such that for all values of the variables $x_1, \dots, x_n$ we have that $C[g(e_1, \dots, e_k)]$ is equivalent to $C[expr]$ w.r.t. program P ;

—we find recursive equations for $g(x_1, \dots, x_k)$; and

—we replace the equation $f(x_1, \dots, x_n) = C[expr]$ by the following one:

$$f(x_1, \dots, x_n) = C[g(e_1, \dots, e_k)].$$

- (3) *Lambda Abstraction* [Pettorossi and Skowron 1987; Pettorossi 1987b]. It consists of replacing a given expression e_1 by $e_1[x/e_2]$ **where** $x = e_2$, or equivalently, $(\lambda x. e_1[x/e_2]) e_2$, where x is a fresh new variable and e_2 is a subexpression of e_1 . Lambda Abstraction is a form of generalization that can be applied when an expression and one of its subexpressions both visit the same data structure and that subexpression determines a mismatch that makes it impossible to perform a folding step (see Example 2.3.6).

2.3 Examples of Transformations of Functional Programs

We now give some examples of applications of the rules and strategies we have introduced.

Example 2.3.1 (The Composition Strategy: List Processing). The following program computes the sum of the even elements of a list, say l , of natural numbers.

$$\text{evensum}(l) = \text{sum}(\text{even}(l)) \quad (1)$$

$$\text{even}([]) = [] \quad (2)$$

$$\text{even}(a:l) = \text{if odd}(a) \text{ then } \text{even}(l) \text{ else } a:\text{even}(l) \quad (3)$$

$$\text{sum}([]) = 0 \quad (4)$$

$$\text{sum}(a:l) = a + \text{sum}(l). \quad (5)$$

Now, by using the composition strategy we can avoid the construction of the intermediate list of the even elements of l , which is the value of $\text{even}(l)$ and it is passed as input to $\text{sum}(_)$. We compose the functions $\text{sum}(_)$ and $\text{even}(_)$ and we define the new function:

$$h(l) = \text{sum}(\text{even}(l)).$$

Thus

$$h([]) = \text{sum}(\text{even}([])) = \{\text{unfolding}\} = \text{sum}([]) = \{\text{unfolding}\} = 0 \quad (6)$$

$$\begin{aligned} h(a:l) &= \text{sum}(\text{even}(a:l)) = \{\text{unfolding}\} \\ &= \text{sum}(\text{if odd}(a) \text{ then } \text{even}(l) \text{ else } a:\text{even}(l)) \\ &= \{\text{distributivity of sum w.r.t. if-then-else}\} \\ &= \text{if odd}(a) \text{ then } \text{sum}(\text{even}(l)) \text{ else } \text{sum}(a:\text{even}(l)) = \{\text{unfolding}\} \\ &= \text{if odd}(a) \text{ then } \text{sum}(\text{even}(l)) \text{ else } a + \text{sum}(\text{even}(l)) = \{\text{folding}\} \\ &= \text{if odd}(a) \text{ then } h(l) \text{ else } a + h(l). \end{aligned} \quad (7)$$

We finally express evensum in terms of the composite function h . In our case from Equation (1) we simply have: $\text{evensum}(l) = h(l)$.

The various transformation steps of the derivation of Equation (7) for $h(_)$ have been suggested by the so-called *need-for-folding* (see Darlington [1981], where it is called *forced folding*), that is, the requirement of making recursive calls to the composite function $h(_)$, rather than to the component functions $\text{sum}(_)$ and $\text{even}(_)$.

In particular, the use of the distributivity of sum w.r.t. **if-then-else** was suggested by the syntactic requirement of producing the occurrences of $\text{sum}(\text{even}(l))$ which can then be folded using $h(l)$. By performing those folding steps, the efficiency improvements due to the unfolding steps that lead from the expression “ $\text{sum}(\text{even}(a:l))$ ” to the expression “**if odd}(a) \text{ then } \text{sum}(\text{even}(l)) \text{ else } a + \text{sum}(\text{even}(l))**”, can be iterated *at each level of recursion*, and thus they become computationally significant. Indeed, in what follows we see that by performing folding steps we may transform an exponential time algorithm into a linear time one.

The idea of need-for-folding plays a major role in the program transformation methodology. It can be regarded as a metastrategy because, as shown in the

following examples, it is the need-for-folding that often suggests the suitable strategies to apply.

Example 2.3.2 (The Tupling Strategy: Towers of Hanoi). Let us consider the following problem. There are three pegs: A , B , and C , and there are n disks, with $n \geq 0$, of different sizes that are stacked as a tower on peg A with smaller disks on top of larger disks. It is required to move the disks from peg A to peg B using peg C as an auxiliary peg. It is possible to move one disk at a time only, and that disk has to be the smallest one of both the tower it comes from and the tower to which it goes.

Let M be the set of possible elementary moves, that is, $M = \{AB, BC, CA, BA, CB, AC\}$, where XY denotes the move that takes the top disk from peg X to the top of peg Y . Let M^* be the free monoid of sequences of elementary moves. The constant *skip* denotes the empty sequence of moves, and “ $::$ ” denotes the associative concatenation of sequences. A solution to the Towers of Hanoi problem is provided by the function $f: N \times Pegs^3 \rightarrow M^*$, where N is the set of natural numbers and $Pegs$ is the set $\{A, B, C\}$, defined as follows:

$$f(0, a, b, c) = skip, \quad (8)$$

$$f(n+1, a, b, c) = f(n, a, c, b) :: ab :: f(n, c, b, a) \quad \text{for } n \geq 0, \quad (9)$$

where the variables a , b , and c assume different values in $Pegs$.

We derive a solution that does not require the use of a stack and is more efficient than the recursive one. We proceed as follows: (Step α) we transform the general recursion of Equation (9) into a linear recursion, and then (Step β) we transform the linear recursive program into an iterative program. In Step α we make use of the tupling strategy, and in Step β we use a simple schema equivalence.

Step α . The program transformation process begins by performing some unfolding steps. We get, for $n \geq 1$:

$$\begin{aligned} f(n+1, a, b, c) &= (f(n-1, a, b, c) :: ac :: f(n-1, b, c, a)) :: ab :: \\ &\quad (f(n-1, c, a, b) :: cb :: f(n-1, a, b, c)). \end{aligned}$$

Here and in the sequel we avoid the explicit use of the instantiation rule by assuming that given a context $C[\dots]$, the expression “ $f(n) = C[n-k]$ for $n \geq k$ ” stands for “ $f(n+k) = C[n]$ for $n \geq 0$ ”.

Now we have that $f(n-1, a, b, c)$ and $f(n-1, b, c, a)$ require the common subcomputation $f(n-2, a, c, b)$, whereas $f(n-1, b, c, a)$ and $f(n-1, c, a, b)$ require the common subcomputation $f(n-2, b, a, c)$. Thus in order to avoid repeated subcomputations we apply the tupling strategy and we define a new function t as follows:

$$t(n, a, b, c) = \langle f(n, a, b, c), f(n, b, c, a), f(n, c, a, b) \rangle. \quad (10)$$

We then look for the recursive equations of $t(n, a, b, c)$ and we get:

$$t(n+2, a, b, c)$$

$$= \langle f(n+2, a, b, c), f(n+2, b, c, a), f(n+2, c, a, b) \rangle = \{\text{unfolding}\}$$

$$= \langle (f(n, a, b, c) :: ac :: f(n, b, c, a)) :: ab :: (f(n, c, a, b) :: cb :: f(n, a, b, c)),$$

$$(f(n, b, c, a) :: ba :: f(n, c, a, b)) :: bc :: (f(n, a, b, c) :: ac :: f(n, b, c, a)),$$

$$\begin{aligned}
& (f(n, c, a, b) :: cb :: f(n, a, b, c)) :: ca :: (f(n, b, c, a) :: ba :: f(n, c, a, b))) \\
& = \{ \text{folding and where-abstraction} \} \\
& = \langle (u :: ac :: v) :: ab :: (w :: cb :: u), \\
& \quad (v :: ba :: w) :: bc :: (u :: ac :: v), \\
& \quad (w :: cb :: u) :: ca :: (v :: ba :: w) \rangle \textbf{ where } \langle u, v, w \rangle = t(n, a, b, c) \quad \text{for } n \geq 0.
\end{aligned} \tag{11}$$

Because $t(n + 2, a, b, c)$ is defined in terms of $t(n, a, b, c)$, in order to ensure the termination of the evaluation of $t(n + 2, a, b, c)$ for $n \geq 0$, we need to provide the equations for $t(0, a, b, c)$ and $t(1, a, b, c)$. Unfolding by using Equations (8)–(10) we get:

$$t(0, a, b, c) = \langle \text{skip}, \text{skip}, \text{skip} \rangle, \tag{12}$$

$$t(1, a, b, c) = \langle ab, bc, ca \rangle. \tag{13}$$

Thus we have obtained a linear recursive program for the function t . We then express the function $f(n, a, b, c)$ in terms of $t(n-2, a, b, c)$ by unfolding, where-abstraction, and folding steps. Recalling that “ $::$ ” is associative, we get:

$$\begin{aligned}
f(n, a, b, c) &= \langle (f(n-2, a, b, c) :: ac :: f(n-2, b, c, a)) :: ab :: (f(n-2, c, a, b) \\
& \quad :: cb :: f(n-2, a, b, c)) \rangle \\
&= \langle u :: ac :: v :: ab :: w :: cb :: u \rangle \\
& \quad \textbf{where } \langle u, v, w \rangle = t(n-2, a, b, c) \quad \text{for } n \geq 2.
\end{aligned} \tag{14}$$

In order to preserve termination, because $f(n, a, b, c)$ is defined in terms of $t(n-2, a, b, c)$, we have to provide the values of $f(0, a, b, c)$ and $f(1, a, b, c)$. The value of $f(0, a, b, c)$ is given by Equation (8). The value of $f(1, a, b, c)$ is obtained from Equations (8) and (9):

$$f(1, a, b, c) = ab. \tag{15}$$

Thus we have derived a linear recursive program for the function f . It is made out of Equations (8) and (11)–(15).

Step β . Now we can transform the linear recursive program we have derived into an iterative program, and thus we avoid the use of a stack.

We apply the schema equivalence that transforms Equations (8), (14), and (15) into the program:

$$\begin{aligned}
& \{ n = N \text{ and } \langle T_1, T_2, T_3 \rangle = t(n-2, a, b, c) \} \\
& \textbf{if } n = 0 \textbf{ then } F := \text{skip} \textbf{ else if } n = 1 \textbf{ then } F := ab \textbf{ else} \\
& F := T_1 :: ac :: T_2 :: ab :: T_3 :: cb :: T_1 \\
& \quad \{ F = f(N, a, b, c) \}.
\end{aligned}$$

We also apply the schema equivalence of Figure 2. This equivalence (which can be proved by induction on the natural number N) is a simple generalization of the one obtained by dropping the constant argument r from the functions t

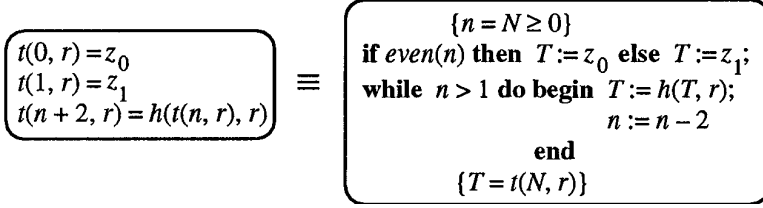


Figure 2 Schema equivalence for transforming linear recursion into iteration.

and h . The matching substitution is given by: $z_0 = \langle \text{skip}, \text{skip}, \text{skip} \rangle$, $z_1 = \langle ab, bc, ca \rangle$, $r = \langle a, b, c \rangle$, and

$$\begin{aligned} \lambda x. h(x, \langle a, b, c \rangle) &= \lambda x. \langle (u :: ac :: v) :: ab :: (w :: cb :: u), (v :: ba :: w) :: bc \\ &\quad :: (u :: ac :: v), (w :: cb :: u) :: ca :: (v :: ba :: w) \rangle \\ &\quad \textbf{where } \langle u, v, w \rangle = x. \end{aligned}$$

We get the iterative program Hanoi listed in the following, where, for $i = 1, 2, 3$, T_i denotes the i th projection of T , and the assignment to T in the body of the **while-do** loop is a parallel assignment; that is, each new projection of T is independently computed starting from the old projections of T .

$\{n = N \geq 0\}$ *Program Hanoi*

if $n = 0$ **then** $F := \text{skip}$ **else if** $n = 1$ **then** $F := ab$ **else**
begin $n := n - 2$; **if** $\text{even}(n)$ **then** $T := \langle \text{skip}, \text{skip}, \text{skip} \rangle$ **else** $T := \langle ab, bc, ca \rangle$;
 while $n > 1$ **do begin** $T := \langle T_1 :: ac :: T_2 :: ab :: T_3 :: cb :: T_1,$
 $T_2 :: ba :: T_3 :: bc :: T_1 :: ac :: T_2,$
 $T_3 :: cb :: T_1 :: ca :: T_2 :: ba :: T_3 \rangle$;
 $n := n - 2$
 end;
 $F := T_1 :: ac :: T_2 :: ab :: T_3 :: cb :: T_1$
end $\{F = f(N, a, b, c)\}$

During the program derivation process we described in Example 2.3.2, many other choices for the eureka function t can be made, because there are many other sets of function calls that have common subcomputations. However, not all choices allow us to derive a linear recursive program. We now present a technique based on the so-called *unfolding tree* (or *tree of recursive calls*), which tells us which functions should be tupled together for achieving linear recursion.

The unfolding tree is constructed from a given function call by performing some unfolding steps and recalling only the recursive function calls with their father-son relationship. We then compress that tree by identifying every two nodes having the same recursive call thereby obtaining the *minimal descent directed acyclic graph*, *m-dag* for short [Bird 1980]. In Figure 3 we have depicted the *m-dag* for $f(n, a, b, c)$ defined by the equation $f(n, a, b, c) = f(n-1, a, c, b) :: ab :: f(n-1, c, b, a)$. The label l in the figure denotes the left call $f(n-1, a, c, b)$ and the label r denotes the right call $f(n-1, c, b, a)$.

Given the minimal descent dag of a recursively defined function, we define an irreflexive and transitive ordering on nodes as follows: for any two nodes m and n ,

we have that $m > n$ holds iff the function call associated with node m requires the computation of the function call associated with node n .

A *cut* in an m -dag is a set of nodes such that if we remove them, with their incoming and outgoing edges, we are left with two disconnected subgraphs g_1 and g_2 such that for any node m in g_1 and any node n in g_2 we have that $m > n$.

A sequence of cuts $\langle c_i | 0 \leq i \rangle$ in an m -dag is said to be *progressive* [Pettorossi 1984] iff

- (i) $i > 0$. c_{i-1} and c_i have the same finite cardinality, and
- (ii) $i > 0$. $c_{i-1} \neq c_i$, and
- (iii) $i > 0$. $n \in c_i$. $m \in c_{i-1}$. if $n \neq m$ then $m > n$, and
- (iv) $i > 0$. $m \in c_{i-1}$. $n \in c_i$. if $n \neq m$ then $m > n$.

From (i) and (ii) it follows that for all $i > 0$ neither c_{i-1} is contained in c_i nor is c_i contained in c_{i-1} . In intuitive terms, while moving along a progressive sequence of cuts from c_{i-1} to c_i we delete “large” nodes and we add “small” nodes. Thus given the m -dag of Figure 3 where $m > n$ is depicted by positioning the node m above the node n , we have, among others, the following cuts:

$$\begin{aligned}
 c_0 &= \{f(n-2, a, b, c), f(n-2, b, c, a), f(n-2, c, a, b)\}, \\
 c_1 &= \{f(n-4, a, b, c), f(n-4, b, c, a), f(n-4, c, a, b)\}, \\
 &\dots, \\
 c_i &= \{f(n-2i-2, a, b, c), f(n-2i-2, b, c, a), f(n-2i-2, c, a, b)\}, \\
 &\dots,
 \end{aligned}$$

and $S = \langle c_0, c_1, \dots, c_i, \dots \rangle$ is a progressive sequence of cuts.

It can be shown that if a progressive sequence of cuts $\langle c_i | 0 \leq i \rangle$ exists for the m -dag of the function call f such that:

- (1) the initial function call of f can be computed from the values of the function calls of the cut c_0 , and
- (2) there exists a function, say h , such that h does not depend on i , and for each $i \geq 0$ the function calls of c_{i-1} can be computed from the function calls of c_i using h ,

then the tupling strategy that tuples together the function calls in a cut generates a linear recursive program for the computation of f .

This result allowed us to perform Step α of the derivation of Example 2.3.2. Indeed, Equation (10) has been determined by tupling the function calls of a cut in the aforementioned sequence S .

With respect to Step β of that derivation, that is, the transformation from a linear recursive program to an iterative program, we recall that there is a general result [Paterson and Hewitt 1970] that ensures that every linear recursion can be translated into an iteration using a constant number of memory cells (i.e., avoiding the use of a stack). However, that result cannot be used in the context of the program transformation methodology, because it degrades the time performance from $O(n)$ to $O(n^2)$. Thus we use, instead, the schema equivalence of Figure 2.

Various techniques for the automation of the tupling strategy are described in Chin [1990; 1993].

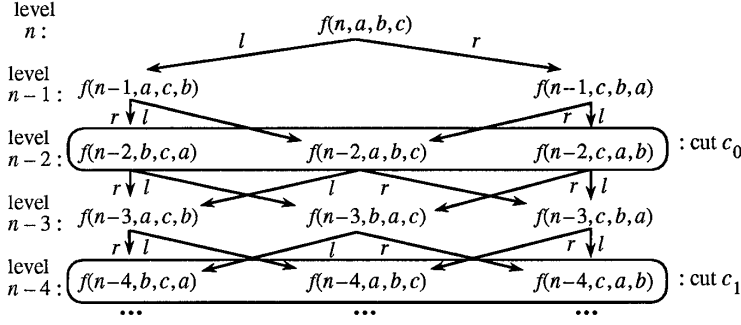
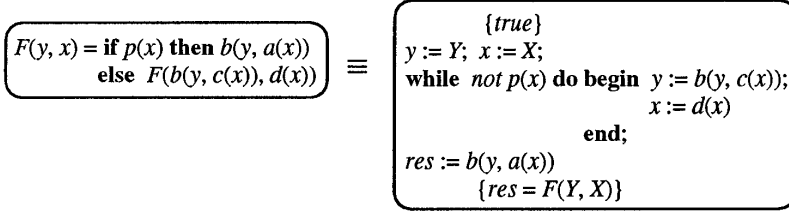
Figure 3 Unfolding tree of $f(n, a, b, c)$.

Figure 4 Schema equivalence for tail recursion.

Example 2.3.3 (The Generalization Strategy from Expressions to Variables: Linear Recursion Without Stack). Let us consider the following linear recursive program:

$$f(x) = \text{if } p(x) \text{ then } a(x) \text{ else } b(c(x), f(d(x))). \quad (16)$$

Let us assume that the function $b(_, _)$ is strict w.r.t. its arguments and $b(_, _)$ is associative. Thus $b(_, _)$ distributes w.r.t. **if-then-else**. In order to improve efficiency when evaluating $f(x)$, we want to derive an iterative program. This can be done in the following steps: (α) we first look for a tail recursive program, and (β) we then apply a schema equivalence to transform this tail recursive program into an iterative one.

Step (α) We have:

$$\begin{aligned}
 f(x) &= \text{if } p(x) \text{ then } a(x) \text{ else } b(c(x), f(d(x))) = \{\text{unfolding}\} \\
 &= \text{if } p(x) \text{ then } a(x) \text{ else} \\
 &\quad b(c(x), \text{if } p(d(x)) \text{ then } a(d(x)) \text{ else } b(c(d(x)), f(d(d(x))))) \\
 &= \{\text{distributivity of } b(_, _) \text{ w.r.t. if-then-else}\} \\
 &= \text{if } p(x) \text{ then } a(x) \text{ else} \\
 &\quad \text{if } p(d(x)) \text{ then } b(c(x), a(d(x))) \text{ else } b(c(x), b(c(d(x)), f(d(d(x))))) \\
 &= \{\text{associativity of } b(_, _)\} \\
 &= \text{if } p(x) \text{ then } a(x) \text{ else} \\
 &\quad \text{if } p(d(x)) \text{ then } b(c(x), a(d(x))) \text{ else } b(b(c(x), c(d(x))), f(d(d(x)))).
 \end{aligned}$$

A tail recursive program can be achieved if we can fold the expression $b(b(c(x), c(d(x))), f(d(d(x))))$, call it e_1 , with the previously derived expression $b(c(d(x)), f(d(d(x))))$, call it e_2 , which is the value of $f(d(x))$ for $p(d(x)) = \text{false}$. The mismatch between the first arguments of the outermost b s in e_1 and e_2 does not allow us to perform a folding step.

Thus the need-for-folding suggests generalizing the first argument of the function b to a variable, say y . By applying the generalization strategy, we define the following function F :

$$F(y, x) = b(y, f(x)), \quad (17)$$

whose r.h.s. is an expression that generalizes both e_1 and e_2 . We then get the following recursive equation for F :

$$\begin{aligned} F(y, x) &= b(y, f(x)) = \{\text{using Equation (16)}\} \\ &= b(y, \text{if } p(x) \text{ then } a(x) \text{ else } b(c(x), f(d(x)))) \\ &= \{\text{distributivity of } b(-, -) \text{ w.r.t. if-then-else}\} \\ &= \text{if } p(x) \text{ then } b(y, a(x)) \text{ else } b(y, b(c(x), f(d(x)))) \\ &= \{\text{associativity of } b(-, -)\} \\ &= \text{if } p(x) \text{ then } b(y, a(x)) \text{ else } b(b(y, c(x)), f(d(x))) = \{\text{folding}\} \\ &= \text{if } p(x) \text{ then } b(y, a(x)) \text{ else } F(b(y, c(x)), d(x)). \end{aligned}$$

Thus by folding Equation (16) using Equation (17) we get:

$$f(x) = \text{if } p(x) \text{ then } a(x) \text{ else } F(c(x), d(x)). \quad (18)$$

Now we have derived the following tail recursive program for computing $f(x)$:

$$f(x) = \text{if } p(x) \text{ then } a(x) \text{ else } F(c(x), d(x)) \quad (18)$$

$$F(y, x) = \text{if } p(x) \text{ then } b(y, a(x)) \text{ else } F(b(y, c(x)), d(x)). \quad (19)$$

Step (β) This tail recursive program can be transformed into an iterative program by applying the schema equivalence shown in Figure 4, which can easily be proved by computational induction.

We finally get the following program:

```

      {x = X}
if p(x) then res := a(x) else
begin y := c(x); x := d(x);
      while not p(x) do begin y := b(y, c(x)); x := d(x) end;
      res := b(y, a(x))
end                {res = f(X)}
```

Now we continue our program derivation by assuming that there exists a neutral element, say η , for $b(-, -)$, that is, $x. b(\eta, x) = x = b(x, \eta)$. We have that:

$$\begin{aligned} f(x) &= \{\text{neutral element}\} = b(\eta, f(x)) \\ &= \{\text{folding using Equation (17)}\} = F(\eta, x). \end{aligned} \quad (18^*)$$

Thus by applying again the schema equivalence of Figure 4 for $Y = \eta$, we get the following iterative program for computing the value of $f(X)$ (18*) that is equal to $F(\eta, X)$:

$$\begin{aligned} & \{true\} \\ y := \eta; x := X; \\ \mathbf{while\ not\ } p(x) \mathbf{\ do\ begin\ } & y := b(y, c(x)); x := d(x) \mathbf{\ end;}\ \\ res := b(y, a(x)) & \\ & \{res = F(\eta, X) = f(X)\} \end{aligned}$$

In particular, the program:

$$fact(x) = \mathbf{if\ } x = 0 \mathbf{\ then\ } 1 \mathbf{\ else\ } x \times fact(x - 1) \quad \text{for } x \geq 0 \quad (20)$$

is an instance of Equation (16) for $p(x) = (x = 0)$, $a(x) = 1$, $b(y, x) = y \times x$, $c(x) = x$, and $d(x) = x - 1$. We have that the neutral element η for $b(_, _)$ is 1. Thus Equations (18*) and (19) produce the following tail recursive program for $fact(x)$:

$$fact(x) = G(1, x) \quad \text{for } x \geq 0 \quad (21)$$

$$G(y, x) = \mathbf{if\ } x = 0 \mathbf{\ then\ } y \mathbf{\ else\ } G(y \times x, x - 1). \quad (22)$$

The corresponding iterative program obtained by again using the schema equivalence of Figure 4, is:

$\{N \geq 0\}$	<i>Program Factorial.1</i>
----------------	----------------------------

$$\begin{aligned} y := 1; \ x := N; \ \mathbf{while\ } x \neq 0 \mathbf{\ do\ begin\ } & y := y \times x; \ x := x - 1 \mathbf{\ end;} \\ res := y \times 1 & \\ & \{res = G(1, N) = fact(N)\}. \end{aligned}$$

where the precondition $\{true\}$ of Figure 4 has been strengthened by $\{N \geq 0\}$. In this last program we can replace $x \neq 0$ by $x > 0$ (because $N \geq 0$) and $y \times 1$ by y . We can also get rid of the variable res , and use, instead, the variable y . Thus the assignment $res := y \times 1$ can be avoided. By performing these simplifications we get:

$\{N \geq 0\}$	<i>Program Factorial.2</i>
----------------	----------------------------

$$\begin{aligned} y := 1; \ x := N; \ \mathbf{while\ } x > 0 \mathbf{\ do\ begin\ } & y := y \times x; \ x := x - 1 \mathbf{\ end;} \\ & \{y = fact(N)\} \end{aligned}$$

The program for the function f consisting of Equations (18*) and (19) can also be derived from Equation (16) by applying the so-called *accumulation strategy* [Bird 1984b], whereby given an associative operation $(b(_, _))$, in our case, we can store the partial results of its application at each recursive call in a new argument (y , in our case). The derivation we have presented in Example 2.3.3 indicates that such a new argument, which plays the role of an accumulator, does not come from an ad hoc invention, but comes from a generalization step.

Now we give some more examples where it is shown that:

—the generalization strategy from (sub)-expressions to variables may allow us to get a tail recursive program, even though the initial program is not linear recursive (Example 2.3.4);

- the generalization from functions to functions by implicit definition may allow us to exponentially reduce the time complexity (Example 2.3.5); and
- the Lambda abstraction strategy allows us to define and manipulate pointers in a disciplined way and visit data structures only once (Example 2.3.6). (For a transformational approach to the derivation of programs that use pointers, the reader may also refer to Möller [1993].)

Example 2.3.4 (The Generalization Strategy from Expressions to Variables: Tree Processing). Let us consider the following program, which computes the height of a binary tree [Chatelin 1976].

$$h(\text{tip}(m)) = 0 \quad (23)$$

$$h(\text{tree}(S, T)) = 1 + \max(h(S), h(T)). \quad (24)$$

We perform some unfolding steps starting from Equation (24) and we have:

$$\begin{aligned} h(\text{tree}(S, T)) &= 1 + \max(h(S), h(T)) \\ &= \max(1 + h(S), 1 + h(T)) = \{\text{assuming } S = \text{tree}(S1, S2)\} \quad (\dagger) \\ &= \max(2 + \max(h(S1), h(S2)), 1 + h(T)) \\ &= \{\text{distributivity of } + \text{ over } \max\} \\ &= \max(\max(2 + h(S1), 2 + h(S2)), 1 + h(T)) \\ &= \{\text{associativity of } \max\} \\ &= \max(2 + h(S1), \max(2 + h(S2), 1 + h(T))). \end{aligned} \quad (25)$$

In order to get a tail recursive program by folding, we match this last expression against a previous r.h.s. of Equation (25) and, in particular, we choose $\max(1 + h(S), 1 + h(T))$ (see $\dagger$). We have that the expression $\max(2 + h(S1), \max(2 + h(S2), 1 + h(T)))$ is an instance of $\max(1 + h(S), 1 + h(T))$ if we generalize the constant 1 (occurring in $1 + h(S)$) and the second argument $1 + h(T)$ to the variables n and u , respectively. Thus by applying the generalization strategy, we introduce the following function g :

$$g(n, S, u) = \max(n + h(S), u). \quad (26)$$

We have that:

$$\begin{aligned} h(T) &= \{\text{properties of } \max\} = \max(0 + h(T), 0) \\ &= \{\text{folding using (26)}\} = g(0, T, 0). \end{aligned} \quad (27)$$

The recursive equations for the function g are as follows:

$$\begin{aligned} g(n, \text{tip}(m), u) &= \{\text{by (26) and (23)}\} = \max(n + 0, u) = \max(n, u) \quad (28) \\ g(n, \text{tree}(S, T), u) &= \max(n + h(\text{tree}(S, T)), u) = \{\text{unfolding}\} \\ &= \max(n + 1 + \max(h(S), h(T)), u) \\ &= \{\text{distributivity of } + \text{ over } \max\} \\ &= \max(\max(n + 1 + h(S), n + 1 + h(T)), u) \end{aligned}$$

$$\begin{aligned}
 &= \{\text{associativity of } \max\} \\
 &= \max(n + 1 + h(S), \max(n + 1 + h(T), u)) \\
 &= \{\text{folding using Equation (29)}\} \\
 &= g(n + 1, S, \max(n + 1 + h(T), u)). \tag{29}
 \end{aligned}$$

By a final folding step using Equation (26) we get:

$$g(n, \text{tree}(S, T), u) = g(n + 1, S, g(n + 1, T, u)). \tag{30}$$

Thus one of the two genuine recursive calls of the function h in the r.h.s. of Equation (24) has been transformed into a tail recursive one.

The final program is given by Equations (27), (28), and (30).

Example 2.3.5 (The Generalization Strategy from Functions to Functions: Linear Recurrence Relations in Logarithmic Time). We first consider the simple case of the Fibonacci function.

Program Fibonacci.0

$$f(0) = 1 \tag{31}$$

$$f(1) = 1 \tag{32}$$

$$f(n + 2) = f(n + 1) + f(n) \quad \text{for } n \geq 0. \tag{33}$$

We first generalize the two occurrences of the constant 1 in the r.h.s. of Equations (31) and (32) to the distinct variables a_0 and a_1 . We get the following equations, where $n \geq 0$:

$$G(a_0, a_1, 0) = a_0 \tag{34}$$

$$G(a_0, a_1, 1) = a_1 \tag{35}$$

$$G(a_0, a_1, n + 2) = G(a_0, a_1, n + 1) + G(a_0, a_1, n) \tag{36}$$

$$f(n) = G(1, 1, n). \tag{37}$$

We then generalize the constant 2 occurring in Equation (36) to a variable, say k , and we define the following function for $n \geq 0$ and $k \geq 0$:

$$F(a_0, a_1, n, k) = G(a_0, a_1, n + k). \tag{38}$$

We have:

$$F(a_0, a_1, n, 0) = G(a_0, a_1, n) \tag{39}$$

$$F(a_0, a_1, n, 1) = G(a_0, a_1, n + 1) \tag{40}$$

$$\begin{aligned}
 F(a_0, a_1, n, k + 2) &= G(a_0, a_1, n + k + 2) = \{\text{unfolding}\} \\
 &= G(a_0, a_1, n + k + 1) + G(a_0, a_1, n + k) = \{\text{folding}\} \\
 &= F(a_0, a_1, n, k + 1) + F(a_0, a_1, n, k). \tag{41}
 \end{aligned}$$

We may then perform some unfolding steps starting from $F(a_0, a_1, n, k + 2)$. We get:

$$\begin{aligned} F(a_0, a_1, n, k + 2) &= F(a_0, a_1, n, k + 1) + F(a_0, a_1, n, k) \\ &= 2F(a_0, a_1, n, k) + F(a_0, a_1, n, k - 1) \\ &= 3F(a_0, a_1, n, k - 1) + 2F(a_0, a_1, n, k - 2), \end{aligned}$$

and eventually we get:

$$\begin{aligned} F(a_0, a_1, n, k + 2) &= c_1 F(a_0, a_1, n, 1) + c_2 F(a_0, a_1, n, 0) = \{\text{unfolding}\} \\ &= c_1 G(a_0, a_1, n + 1) + c_2 G(a_0, a_1, n). \end{aligned}$$

We can now perform a generalization step from functions to functions *by implicit definition* and we assume that the constants c_1 and c_2 are the values of two functions, say $s(k)$ and $r(k)$. Thus we assume that:

$$F(a_0, a_1, n, k) = r(k) G(a_0, a_1, n) + s(k) G(a_0, a_1, n + 1), \quad (42)$$

and we look for the explicit definition of the functions $r(k)$ and $s(k)$.

From Equations (39) and (42) we get $G(a_0, a_1, n) = r(0) G(a_0, a_1, n) + s(0) G(a_0, a_1, n + 1)$, and because this equality should hold for all values of a_0 and a_1 , we get $r(0) = 1$ and $s(0) = 0$.

Analogously, from Equations (40) and (42) we get $r(1) = 0$ and $s(1) = 1$. From the r.h.s. of Equations (41) and (42) (for $k + 2$ instead of k) we get:

$$F(a_0, a_1, n, k + 1) + F(a_0, a_1, n, k) = r(k + 2)G(a_0, a_1, n) + s(k + 2)G(a_0, a_1, n + 1).$$

By Equation (42) for k and $k + 1$ we have:

$$\begin{aligned} r(k + 1)G(a_0, a_1, n) + s(k + 1)G(a_0, a_1, n + 1) \\ + r(k)G(a_0, a_1, n) + s(k)G(a_0, a_1, n + 1) \\ = r(k + 2)G(a_0, a_1, n) + s(k + 2)G(a_0, a_1, n + 1) \end{aligned}$$

which gives us the two equations: $r(k + 2) = r(k + 1) + r(k)$ and $s(k + 2) = s(k + 1) + s(k)$.

Therefore, having $r(0) = 1$, $r(1) = 0$, and $r(k + 2) = r(k + 1) + r(k)$ we derive by Equations (34)–(36) that:

$$r(k) = G(1, 0, k) \quad \text{and analogously,} \quad s(k) = G(0, 1, k).$$

Thus from Equation (42) and Definition (48) we get for $n \geq 0$ and $k \geq 0$:

$$G(a_0, a_1, n + k) = G(1, 0, k)G(a_0, a_1, n) + G(0, 1, k)G(a_0, a_1, n + 1). \quad (43)$$

From Equation (43) for $n = k$ and $n = k + 1$ we get Equations (44) and (45) of the following program for computing the Fibonacci function, where $n \geq 0$ and $k > 0$:

Program Fibonacci.1

$$f(n) = G(1, 1, n) \quad (37)$$

$$G(a_0, a_1, 0) = a_0 \quad (34)$$

$$G(a_0, a_1, 1) = a_1 \quad (35)$$

$$G(a_0, a_1, 2k) = G(1, 0, k) G(a_0, a_1, k) + G(0, 1, k) G(a_0, a_1, k + 1) \quad (44)$$

$$G(a_0, a_1, 2k + 1) = G(1, 0, k) G(a_0, a_1, k + 1) + G(0, 1, k) G(a_0, a_1, k + 2). \quad (45)$$

Despite the fact that at each recursive call the third argument of G is divided by 2 (apart from some additive constants), this program does *not* have a logarithmic running time, because the recursion for G is *not* linear. However, the transformation process may continue by applying the tupling strategy, because the recursive calls of $G(1, 0, k)$ and $G(0, 1, k)$ have the common subcomputation $G(1, 0, k/2)$, and we introduce, by the definition rule, the following function $p(k)$ for $k \geq 0$:

$$p(k) = \langle G(1, 0, k), G(0, 1, k) \rangle.$$

Looking for the recursive equations of $p(k)$ we get:

$$p(0) = \langle 1, 0 \rangle$$

$$p(1) = \langle 0, 1 \rangle$$

$$p(2k) = \langle G(1, 0, 2k), G(0, 1, 2k) \rangle = \{\text{unfolding using Equation (43)}\}$$

$$= \langle G(1, 0, k)G(1, 0, k) + G(0, 1, k)G(1, 0, k + 1),$$

$$G(1, 0, k)G(0, 1, k) + G(0, 1, k)G(0, 1, k + 1) \rangle$$

$$= \{G(1, 0, k + 1) = G(0, 1, k) \text{ and } G(0, 1, k + 1) = G(0, 1, k) + G(1, 0, k)\}$$

$$= \langle a^2 + b^2, b^2 + 2ab \rangle \quad \textbf{where } \langle a, b \rangle = p(k)$$

$$p(2k + 1) = \langle G(1, 0, 2k + 1), G(0, 1, 2k + 1) \rangle = \{\text{unfolding using Equation (43)}\}$$

$$= \langle 2ab + b^2, (a + b)^2 + b^2 \rangle \quad \textbf{where } \langle a, b \rangle = p(k).$$

The computation time of the function p is logarithmic and we achieved our goal. Now we need to express the function f in terms of the function p , instead of G , and we have:

$$f(0) = 1$$

$$f(1) = 1$$

$$f(2k) = G(1, 1, 2k) = \{\text{by Equation (43)}\}$$

$$= G(1, 0, k) G(1, 1, k) + G(0, 1, k) G(1, 1, k + 1) = \{\text{by Equation (43)}\}$$

$$= G(1, 0, k)(G(1, 0, k) + G(0, 1, k)) + G(0, 1, k)(G(1, 0, k) + 2G(0, 1, k))$$

$$= (a + b)^2 + b^2 \quad \textbf{where } \langle a, b \rangle = p(k)$$

$$f(2k + 1) = G(1, 1, 2k + 1) = \{\text{analogously to the case for } f(2k)\}$$

$$= (a + b)^2 + 2b(a + b) \quad \textbf{where } \langle a, b \rangle = p(k).$$

We finally get the following program, where $k > 0$:

$f(0) = 1$	<i>Program Fibonacci.2</i>
$f(1) = 1$	
$f(2k) = (a + b)^2 + b^2$	where $\langle a, b \rangle = p(k)$
$f(2k + 1) = (a + b)^2 + 2b(a + b)$	where $\langle a, b \rangle = p(k)$
$p(0) = \langle 1, 0 \rangle$	
$p(1) = \langle 0, 1 \rangle$	
$p(2k) = \langle a^2 + b^2, 2ab + b^2 \rangle$	where $\langle a, b \rangle = p(k)$
$p(2k + 1) = \langle 2ab + b^2, (a + b)^2 + b^2 \rangle$	where $\langle a, b \rangle = p(k)$.

The derivation of this program for the logarithmic evaluation of the Fibonacci function shows the strength of the combined use of the generalization strategy together with the tupling strategy. In what follows we see more examples of this synergism between the two strategies.

A final derivation step consists of finding an iterative program for computing the Fibonacci function. We can apply the schema equivalence given in Figure 5, whose correctness can easily be proved by induction on the natural number K .

In our case, because the b and d operations are integer divisions by 2, the elements in the stack before (and after) performing the assignment $P := e$ give us the binary expansion “ $k(0), \dots, k(s)$ ” of the number K , if we take $A = 0$ and $C = 1$. The top of the stack is the most significant bit $k(s)$. We may assume that the empty stack represents the binary expansion of 0.

From Program Fibonacci.2 we easily get the following iterative program, where the assignments to u and v are parallel assignments:

$\{N \geq 0\}$	<i>Program LogFibonacci</i>
----------------	-----------------------------

```

if  $N \leq 1$  then  $F := 1$  else
  begin  $K := N \text{ div } 2$ ;  $s := \lfloor \log_2(K) \rfloor$ ;
     $\{K = \sum_{i=0}^s k(i) 2^i\}$ 
     $u := 1$ ;  $v := 0$ ;  $i := s$ ;
    while  $i \geq 0$  do begin if  $k(i) = 0$  then  $u, v := u^2 + v^2, 2uv + v^2$ ;
      if  $k(i) = 1$  then  $u, v := 2uv + v^2, (u + v)^2 + v^2$ ;
       $i := i - 1$ 
    end;
    if even( $N$ ) then  $F := (u + v)^2 + v^2$  else  $F := (u + v)^2 + 2v(u + v)$ 
  end
   $\{F = f(N)\}$ 

```

We can extend the derivation shown in Example 2.3.5 to any homogeneous linear recurrence relation of order r with constant coefficients in any semiring

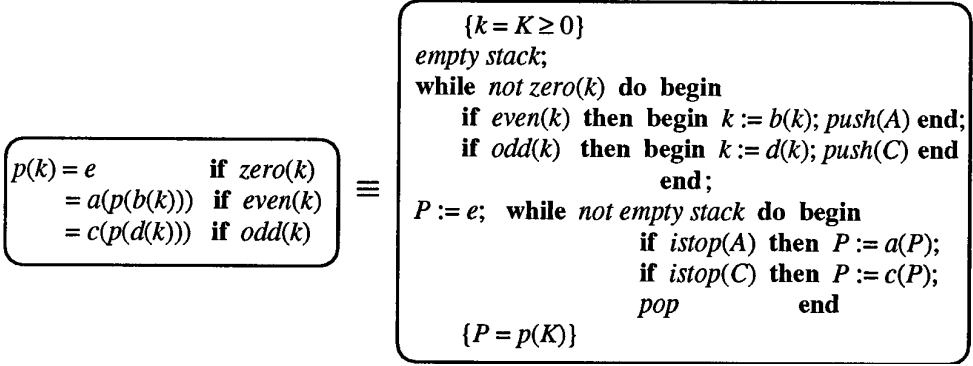


Figure 5 Schema equivalence for linear recursion on natural numbers.

structure whose operations are denoted by $+$ and $\times$. Hence we can derive algorithmic time algorithm for computing any function h defined as follows:

$$h(0) = h_0$$

...

$$h(r-1) = h_{r-1}$$

$$h(n) = b_0 \times h(n-r) + \dots + b_{r-1} \times h(n-1) \quad \text{for } n \geq r.$$

The interested reader may refer to Pettorossi and Burstall [1982].

Example 2.3.6 (The Lambda Abstraction Strategy: Palindrome in One Visit Avoiding the Append Function). Let us consider the following program for testing whether a given list l is palindrome.

$$\text{palindrome}(l) = \text{eqlist}(l, \text{rev}(l)) \quad (46)$$

$$\text{eqlist}([], []) = \text{true} \quad (47)$$

$$\text{eqlist}(a:l_1, l) = (a = \text{hd}(l)) \text{ and } \text{eqlist}(l_1, \text{tl}(l)) \quad (48)$$

$$\text{rev}([]) = [] \quad (49)$$

$$\text{rev}(a:l) = \text{rev}(l) :: [a] \quad (50)$$

$$x :: y = \text{if } x = [] \text{ then } y \text{ else } \text{hd}(x) : (\text{tl}(x) :: y) \quad (51)$$

where $:$ and $::$ are the infix operators for *cons* and *append*, respectively. This program visits the given list twice: a first time for its reversal (when computing *rev*) and a second time for testing equality (when computing *eqlist*). We look for a program that visits the given list only once. We also want to avoid the use of the *append* function which is expensive, because using Equation (51), the number of *cons* operations needed for reversing a list of length n is $O(n^2)$.

Equations (49) and (50) can be transformed into a tail recursive form as indicated in Example 2.3.3. Indeed, they are an instance of Equation (16) for $p(x) = \text{null}(x)$ (i.e., $p(x)$ is true iff $x = []$), $a(x) = \eta = []$, $b(x, y) = y :: x$, $c(x) = [\text{hd}(x)]$, and $d(x) = \text{tl}(x)$. By writing $h(x, y)$ instead of $F(y, x)$, from Equations (18*)

and (19) we get:

$$rev(l) = h(l, []) \quad (52)$$

$$h([], x) = x \quad (53)$$

$$h(a:l, x) = h(l, [a]::x) = h(l, a:x). \quad (54)$$

By Equation (17) we also have: $h(l, x) = F(x, l) = b(x, f(l)) = rev(l)::x$. Thus we obtain a program without the *append* function by replacing the Equations (49)–(51) by the Equations (52)–(54). Then, in order to derive a program that goes through the input list only once, we continue our program transformation by applying the composition strategy. We get:

$$palindrome([]) = eqlist([], rev([])) = \{\text{unfolding}\} = true \quad (55)$$

$$\begin{aligned} palindrome(a:l) &= eqlist(a:l, rev(a:l)) = \{\text{unfolding}\} \\ &= (a = hd(rev(a:l))) \text{ and } eqlist(l, tl(rev(a:l))). \end{aligned} \quad (56)$$

When trying to fold the r.h.s. of Equation (56) using Equation (46) we have a mismatch between the expression $eqlist(l, rev(l))$ in Equation (46) and $eqlist(l, tl(rev(a:l)))$ in Equation (56). We can apply the Lambda Abstraction strategy which consists in abstracting away the mismatching argument. In our case we rewrite Equation (56) as follows:

$$\begin{aligned} palindrome(a:l) &= eqlist(a:l, rev(a:l)) = \{\text{by Lambda Abstraction}\} \\ &= (\lambda x. eqlist(a:l, x)) rev(a:l) = \{\text{Equation (52)}\} \\ &= (\lambda x. eqlist(a:l, x)) h(a:l, []). \end{aligned} \quad (57)$$

Now, both $\lambda x. eqlist(a:l, x)$ and $h(a:l, [])$ visit the same data structure $a:l$. We can use the tupling strategy and we define:

$$Q(l) = \langle \lambda x. eqlist(l, x), h(l, []) \rangle \quad (58)$$

whose recursive equations are:

$$\begin{aligned} Q([]) &= \langle \lambda x. eqlist([], x), h([], []) \rangle = \{\text{unfolding}\} = \langle \lambda x. null(x), [] \rangle \\ Q(a:l) &= \langle \lambda x. eqlist(a:l, x), h(a:l, []) \rangle = \{\text{unfolding}\} \\ &= \langle \lambda x. (a = hd(x)) \text{ and } eqlist(l, tl(x)), h(l, [a]) \rangle. \end{aligned}$$

We cannot fold this last equation using Equation (58) because of the mismatch between $h(l, [])$ and $h(l, [a])$. Thus we generalize in the definition of the function $Q(l)$ the list $[]$ to a variable, say y , and we define the tupled function:

$$R(l, y) = \langle \lambda x. eqlist(l, x), h(l, y) \rangle, \quad (59)$$

whose recursive equations are:

$$R([], y) = \langle \lambda x. eqlist([], x), h([], y) \rangle = \langle \lambda x. null(x), y \rangle \quad (60)$$

$$\begin{aligned} R(a:l, y) &= \langle \lambda x. a = hd(x) \text{ and } eqlist(l, tl(x)), h(l, a:y) \rangle = \{\text{folding}\} \\ &= \langle \lambda x. a = hd(x) \text{ and } u(tl(x)), v \rangle \quad \textbf{where } \langle u, v \rangle = R(l, a:y). \end{aligned} \quad (61)$$

We finally express Equation (57) in terms of the components of the function R , thereby getting Equation (57*). We have the following:

Program Palindrome.1

$$palindrome([]) = true \quad (56)$$

$$palindrome(a:l) = u(v) \quad \textbf{where } \langle u, v \rangle = R(a:l, []) \quad (57^*)$$

$$R([], y) = \langle \lambda x. null(x), y \rangle \quad (60)$$

$$R(a:l, y) = \langle \lambda x. a = hd(x) \text{ and } u(tl(x)), v \rangle \quad \textbf{where } \langle u, v \rangle = R(l, a:y). \quad (61)$$

This final algorithm visits the input list only once because the recursive definition of $R(a:l, y)$ is in terms of $R(l, y)$ only.

Notice that if instead of Lambda Abstraction, we apply generalization from expressions to variables, we should introduce the function $S(x, l, y) = \langle eqlist(l, x), h(l, y) \rangle$, instead of $R(l, y)$. However, by doing so we would not be able to express *palindrome* in terms of $S(x, l, y)$.

We can further improve the linear recursive program *Palindrome.1* that we have derived by transforming it into an iterative program as follows. From Equation (46) we have:

$$\begin{aligned} palindrome(a:l) &= eqlist(a:l, rev(a:l)) = \{\text{commutativity of eqlist}\} \\ &= eqlist(rev(a:l), a:l) = \{\text{folding using Equation (59)}\} \\ &= \pi_1(R(rev(a:l), y))(a:l) \text{ for some } y, \end{aligned} \quad (62)$$

where, as usual, $\pi_1(\langle x, y \rangle) = x$.

From Equation (61) we also have that $\pi_1(R(a:l, _))$ does not depend on the value of $\pi_2(R(l, _))$. Thus, when computing $palindrome(a:l)$ according to the preceding Program *Palindrome.1*, we do not need the second projection of $R(rev(a:l), _)$.

Moreover, in the recursive definition of R we have that the second argument of R is only used for modifying itself. This means that there exists a function T such that $l, y.T(l) = \pi_1(R(l, y))$. Indeed, we have: $T(l) = \lambda x. eqlist(l, x)$.

Thus from Equation (62) we get:

$$palindrome(a:l) = T(rev(a:l))(a:l). \quad (62^*)$$

By using Equations (60) and (61) we derive the following program:

Program Palindrome.2

$$palindrome([]) = true \quad (56)$$

$$palindrome(a:l) = T(rev(a:l))(a:l) \quad (62^*)$$

$$T([]) = \lambda x. null(x) \quad (63)$$

$$T(a:l) = \lambda x. a = hd(x) \text{ and } T(l)(tl(x)). \quad (64)$$

We can then apply the schema equivalence of Figure 6, which can be proved by induction on l , and we derive the iterative program *ItPalindrome*.

Program ItPalindrome

$$\{l = L\}$$

if $l = []$ **then** $P := true$ **else**

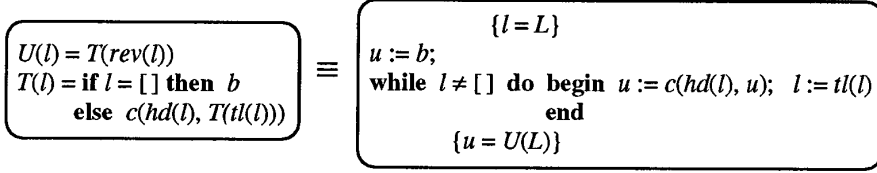


Figure 6 Schema equivalence for linear recursion on lists.

```

begin initl := l;  u :=  $\lambda x.null(x)$ ;
      while l ≠ [] do begin u :=  $\lambda x.(hd(l) = hd(x) \text{ and } u(tl(x)))$ ;  l := tl(l) end;
      P := u(initl)
end                                     {P = palindrome(L)}

```

The final assignment $P := u(\text{initl})$ in the program ItPalindrome could be considered as a second visit of the given list l , and it may seem a bit strange to claim that we have obtained an algorithm that visits the input list only once. However, a detailed analysis [Pettorossi and Proietti 1987] shows that the number of *hd* and *tl* operations used by the ItPalindrome program is reduced by one-third with respect to the naive algorithm (i.e., Equations (46)–(48) and (52)–(54)) that makes two visits of the list l .

The ItPalindrome program also indicates that via program transformation we can discover quite intricate algorithms. Indeed, when developing an algorithm that checks whether a given list is palindrome in one visit, we face the main difficulty of discovering the middle element of the list which is known only when the visit is completed. That difficulty can be overcome by using a stack of pointers in a sophisticated way. Our program transformation shows that this clever use of pointers can be automatically derived by Lambda Abstraction using bound variables within suitable lambda expressions.

3. TRANSFORMATION RULES AND STRATEGIES FOR LOGIC PROGRAMS

In this section we present the “rules + strategies” approach to program transformation in the case of logic programs. Logic programs compute relations rather than functions, and the *nondeterminism* inherent in relations does affect the various transformation techniques we have presented in the previous section in the case of functional programs. We face new problems, but the key ideas of tupling, generalization, and need-for-folding still play an essential role and turn out to be very effective.

An interesting approach to program development and transformation in the presence of nondeterminism is also given in Möller [1991].

The syntax and the semantics of logic programs are defined as follows. We assume that we are given a first order language L made out of a finite set of *function symbols* with arities (including at least one *constant*, that is, a 0-ary function symbol), a finite set of *predicate symbols*, and a countably infinite set of *variable symbols*. Function and predicate symbols are denoted by lower case letters, and variables are denoted by upper case letters.

Given a predicate symbol p of arity n and the terms $t_1, \dots, t_n$ built out of function and variable symbols, we say that $p(t_1, \dots, t_n)$ is an *atom*. A *goal* is a (possibly empty) conjunction of atoms.

A (*definite*) *clause*, say C , is a formula of the following form:

$$X_1, \dots, X_k.(A_1 \wedge \dots \wedge A_m \rightarrow H)$$

also written as

$$H \leftarrow A_1, \dots, A_m,$$

where (1) $X_1, \dots, X_k$ are the variables occurring in $A_1 \wedge \dots \wedge A_m \rightarrow H$; (2) $A_1, \dots, A_m$ with $m \geq 0$, is a goal, called the *body* of the clause and denoted by $bd(C)$; and (3) H is an atom, called the *head* of the clause and denoted by $hd(C)$. By consistently changing the names of the variables in a clause we get a *variant* of that clause. A (*definite*) *logic program* is a conjunction of clauses.

Given a term t we denote by $vars(t)$ the set of variables occurring in t . The same notation is also used for the variables occurring in atoms, goals, and clauses. A term (or an atom) is said to be *ground* if no variable occurs in it. Given a clause C of the form $H \leftarrow A_1, \dots, A_m, B_1, \dots, B_n$, the *linking* variables of $A_1, \dots, A_m$ in C are those occurring in $A_1, \dots, A_m$ and also in $H, B_1, \dots, B_n$.

The *Herbrand universe* H_L associated with L , is the set of terms built out of the function symbols of L . The *least Herbrand model* $M(P)$ of a program P is the following set of ground atoms:

$$M(P) = \{p(t_1, \dots, t_n) \mid$$

p is a predicate symbol in L ,

$$\{t_1, \dots, t_n\} \subseteq H_L,$$

and $P \models p(t_1, \dots, t_n)\}$.

For any given predicate symbol p occurring in L , $M(p, P)$ is the subset of $M(P)$ consisting of atoms with predicate p . Two programs P_1 and P_2 are said to be *equivalent* w.r.t. the predicate p iff $M(p, P_1) = M(p, P_2)$.

We assume that the reader has some familiarity with the operational semantics of logic programs based on SLD-resolution [Lloyd 1987].

3.1 Transformation Rules for Logic Programs

We now introduce the transformation rules used for obtaining new logic programs from old ones.

(1) *Definition Rule*. It consists of introducing a new clause that defines a new predicate in terms of already existing predicates. Clauses introduced by the definition rule are called *definition clauses*.

(2) *Unfolding Rule*. It consists of performing a resolution step. There are the following main differences with respect to the unfolding rule in the functional case.

- (a) The resolution step produces a *unifying* substitution (not a *matching* substitution, as it happens in a rewriting step for functional programs). Thus when we unfold a clause C w.r.t. an atom A in $bd(C)$ using a clause D in the current program, we replace A by $bd(D)$ and we then apply to the resulting clause the substitution that unifies A and $hd(D)$.
- (b) In the functional case we have assumed that the equations defining a program are mutually exclusive. Thus by unfolding a given equation we may get at most one new equation. In contrast, in the logical case there may be several

clauses whose heads are unifiable with an atom A in the body of a clause C . As a result, by unfolding C w.r.t. A using the clauses of the current program, we may obtain several clauses, say $C_1, \dots, C_n$. By an application of the unfolding rule we replace C by (the conjunction of) all clauses $C_1, \dots, C_n$.

(3) *Folding Rule*. It consists of replacing a subgoal, say B , of the body of a clause, say C , such that B is an instance of the body of a definition clause, say D , by the corresponding instance of the head of D .

Similarly to the functional case, we also require that an application of the folding rule be the inverse of an application of the unfolding rule, in the sense that if a clause F is derived by folding clause C using the definition clause D , then by unfolding F using D we may get back (a variant of) clause C .

There are some pitfalls to be avoided when applying the folding rule. For instance, we cannot fold clause $C: p(X) \leftarrow q(t(X))$ using $D: r \leftarrow q(Y)$, even though the body of C is an instance of the body of D . Indeed, by replacing $q(t(X))$ by the head r of D we get the clause $p(X) \leftarrow r$, from which by unfolding we get $p(X) \leftarrow q(Y)$ which is different from clause C .

Notice that if instead of clause D we consider the clause $E: r(Y) \leftarrow q(Y)$, where the variable Y is an argument of the head predicate, then it is possible to fold clause C using E . In general, it is the case that by considering extra variables as arguments of the head predicate we can perform folding steps that otherwise are impossible.

(4) *Goal Replacement Rule*. Let P be a program, C a clause in P , and G_1 a goal occurring in the body of C . Suppose that for some goal G_2 the following formula is true in the least Herbrand model $M(P)$ of P :

$$X_1, \dots, X_k. \quad (Y_1, \dots, Y_m.G_1 \leftrightarrow Z_1, \dots, Z_n.G_2),$$

where: $X_1, \dots, X_k$ are the linking variables of G_1 in C ; $\{Y_1, \dots, Y_m\} = \text{vars}(G_1) - \{X_1, \dots, X_k\}$; $\{Z_1, \dots, Z_n\} = \text{vars}(G_2) - \{X_1, \dots, X_k\}$; and $\{Z_1, \dots, Z_n\} \cap \text{vars}(C) = \{\}$. Then in the body of C we may replace G_1 by G_2 .

(5) *Clause Deletion Rule*. Let P be a program and C a clause in P . We get a new program Q by deleting C from P if C is true in $M(Q)$. We may apply clause deletion in the following cases: $bd(C)$ contains an atom that is not unifiable with the head of any other clause in P ; and C is *subsumed* by another clause D in P , that is, there exists a substitution θ such that $hd(D)\theta = hd(C)$ and $bd(D)\theta$ occurs in $bd(C)$.

The application of the transformation rules preserves *soundness*, in the sense that if we derive program P_2 from program P_1 , then for each predicate p in P_1 we have that: $M(p, P_2) \subseteq M(p, P_1)$. By forcing some restrictions on the use of those transformation rules [Tamaki and Sato 1984], we also preserve *completeness*; that is, we get $M(p, P_2) \supseteq M(p, P_1)$.

The reader may easily verify that the use of the rules in the examples of program transformation we present in this section preserves both soundness and completeness.

Variants of the preceding transformation rules can be shown to be correct w.r.t. richer logic languages and other program semantics (see, for instance, Aravindan and Dung [1993], Bossi and Cocco [1993], Bossi et al. [1992], Gardner and Shepherdson [1991], Kanamori and Horiuchi [1987], Kawamura and Kanamori [1988], Maher [1993], Proietti and Pettorossi [1991a], Sato [1992], Seki [1991, 1993], and Pettorossi and Proietti [1994] for an overview).

3.2 Transformation Strategies for Logic Programs

Here we list some of the strategies we use for transforming logic programs.

Predicate Tupling Strategy. This strategy, also called *tupling*, for short, consists of selecting some atoms, say $A_1, \dots, A_n$, with $n \geq 1$, occurring in the body of a clause C . We introduce a new predicate *newp* defined by a clause T of the form:

$$\text{newp}(X_1, \dots, X_k) \leftarrow A_1, \dots, A_n,$$

where $X_1, \dots, X_k$ are the linking variables of $A_1, \dots, A_n$ in C . We then look for the recursive definition of the predicate *newp* by performing some unfolding, goal replacement, and clause deletion steps followed by some folding steps using clause T .

We finally fold the atoms $A_1, \dots, A_n$ in the body of clause C using clause T .

The tupling strategy is often applied when $A_1, \dots, A_n$ share some variables. The program improvements that can be achieved by using this strategy in the case of logic programs are similar to those that can be achieved by using the composition and/or the tupling strategies in the case of functional programs. In particular, we need to evaluate only once the subgoals common to the computations evoked by the tupled atoms $A_1, \dots, A_n$. We can also avoid multiple visits of data structures and the construction of intermediate bindings [Debray 1988; Proietti and Pettorossi 1991b].

In addition, by using the tupling strategy, sometimes we can obtain a new program version that simulates the execution of the initial program according to a computation rule different from the “left-to-right” computation rule used by Prolog. An automated technique that applies this transformation strategy for reducing the nondeterminism of Prolog programs is the *Compiling Control* technique [Bruynooghe et al. 1989].

As in the functional case, the folding steps needed for deriving the recursive definition of the predicate *newp* can often be performed only if we introduce some new predicates by means of definition clauses.

In the sequel we consider the following three strategies that can be used for introducing those new predicates: the *loop absorption*, the *generalization*, and the *implicit definition*.

In order to describe those strategies we represent the process of performing unfolding and goal replacement steps starting from a clause, say R , as a tree of clauses, called the *unfolding tree*. The root of the unfolding tree is clause R , and for each clause derived by applying either unfolding or goal replacement to a clause N of the unfolding tree we construct a new son of N . In an unfolding tree we also have the usual relations of descendant clause and ancestor clause.

Loop Absorption Strategy. Suppose that a clause C in an unfolding tree has the form: $H \leftarrow A_1, \dots, A_m, B_1, \dots, B_n$, and the body of a descendant D of C contains an instance of $A_1, \dots, A_m$. Suppose also that the clauses in the path from C to D have been generated by applying no transformation rule to $B_1, \dots, B_n$.

The loop absorption strategy is applied by introducing a new predicate defined by the following clause A :

$$\text{newp}(X_1, \dots, X_k) \leftarrow A_1, \dots, A_m,$$

where $\{X_1, \dots, X_k\}$ is the minimum subset of $\text{vars}(A_1, \dots, A_m)$ which is necessary to fold both C and D using a clause whose body is $A_1, \dots, A_m$. We then look for the recursive definition of the predicate *newp*.

The loop absorption strategy is formally introduced in Proietti and Pettorossi [1990]

and more details may be found in that paper. Similar strategies are used in some other techniques for program transformation, such as the Partial Evaluation technique (see Section 4) and the aforementioned Compiling Control.

Generalization Strategy. Given a clause C of the form: $H \leftarrow A_1, \dots, A_m, B_1, \dots, B_n$, we define a new predicate *genp* by a clause G of the form:

$$\text{genp}(X_1, \dots, X_k) \leftarrow \text{Gen}A_1, \dots, \text{Gen}A_m,$$

where $(\text{Gen}A_1, \dots, \text{Gen}A_m)\theta = A_1, \dots, A_m$, for a given substitution θ , and $X_1, \dots, X_k$ are the variables necessary to fold C using G . We then fold C using G and we get:

$$H \leftarrow \text{genp}(X_1, \dots, X_k)\theta, B_1, \dots, B_n.$$

We finally look for the recursive definition of the predicate *genp*.

As already mentioned when describing the transformation strategies of functional programs, the generalization strategy originated in the area of automated theorem proving [Boyer and Moore 1975] and it has been used in a number of program transformation techniques.

A suitable form of the clause G introduced by generalization can often be obtained by matching clause C against one of its descendants, say D , in the unfolding tree considered during program transformation. In particular, we consider the case where:

- (1) D is the clause $K \leftarrow E_1, \dots, E_m, F_1, \dots, F_r$, and D has been obtained from C by applying no transformation rule to $B_1, \dots, B_n$;
- (2) for $i = 1, \dots, m$ the atom E_i has the same predicate of A_i ;
- (3) for $i = 1, \dots, m$ E_i is not an instance of A_i ;
- (4) the goal $\text{Gen}A_1, \dots, \text{Gen}A_m$ is the most concrete generalization of both $A_1, \dots, A_m$ and $E_1, \dots, E_m$; and
- (5) $\{X_1, \dots, X_k\}$ is the minimum subset of $\text{vars}(\text{Gen}A_1, \dots, \text{Gen}A_m)$ which is necessary to fold both C and D using a clause whose body is $\text{Gen}A_1, \dots, \text{Gen}A_m$.

In our logical language we do not have abstraction operators, and thus we cannot consider techniques similar to the Lambda Abstraction strategy that has been presented in the case of functional programs (see Section 2). However, we show in Example 3.3.2 that the effects of abstractions may sometimes be simulated in logic programming by exploiting the power of the unification mechanism.

A strategy analogous to the generalization from functions to functions by implicit definition presented in Section 2 can be obtained as a particular case of the following one [Proietti and Pettorossi 1994].

Implicit Definition Strategy. Let P be a program and C one of its clauses of the form: $H \leftarrow A_1, \dots, A_m, G_1$, where G_1 is a goal made out of one or more atoms. Suppose that we want to replace G_1 by a goal of the form: “ $G_2, \text{impl}(\dots)$ ”, where *impl* is a new predicate symbol and the arguments of *impl* are variables. In order to do so, according to the Goal Replacement Rule we have to look for a set *Expl* of clauses such that in the least Herbrand model $M(P \wedge \text{Expl})$ the following formula holds:

$$X_1, \dots, X_k. (Y_1, \dots, Y_m. G_1 \leftrightarrow Z_1, \dots, Z_n. (G_2, \text{impl}(\dots))), \quad (65)$$

where: $X_1, \dots, X_k$ are the linking variables of G_1 in C ; $\{Y_1, \dots, Y_m\} = \text{vars}(G_1) - \{X_1, \dots, X_k\}$; $\{Z_1, \dots, Z_n\} = \text{vars}(G_2, \text{impl}(\dots)) - \{X_1, \dots, X_k\}$; and $\{Z_1, \dots, Z_n\} \cap \text{vars}(C) = \{\}$.

The set *Expl* provides an explicit definition of the predicate *impl* which is implicitly defined by Formula (65). A practical method to find the set *Expl* consists of the following steps:

Step 1. We consider two different definitions of a new predicate, say *newp*, by introducing two clauses with body G_1 and $(G_2, \text{impl}(\dots))$, respectively, that is:

$$D_1. \text{ newp}(X_1, \dots, X_k) \leftarrow G_1;$$

$$D_2. \text{ newp}(X_1, \dots, X_k) \leftarrow G_2, \text{impl}(\dots).$$

Step 2. We look for the recursive definition of *newp* by using the program $P \wedge D_1$ thereby producing a program Q_1 equivalent to $P \wedge D_1$.

Step 3. We then unfold every occurrence of *newp* in Q_1 by using clause D_2 and get a program Q_2 .

Step 4. We finally obtain the set *Expl* by requiring that by unfolding the occurrences of $\text{impl}(V_1, \dots, V_h)$ in $P \wedge D_2$ using the clauses in *Expl* we derive program Q_2 .

Notice that the validity of Formula (65) in $M(P \wedge \text{Expl})$ follows from the fact that both programs $(P \wedge D_1 \wedge \text{Expl})$ and $(P \wedge D_2 \wedge \text{Expl})$ can be transformed into $(Q_1 \wedge \text{Expl})$, and therefore the two definitions of *newp* are equivalent. Indeed, by Step 2 we have that $(P \wedge D_1 \wedge \text{Expl})$ can be transformed into $(Q_1 \wedge \text{Expl})$. On the other hand, by Step 4 we have that $(P \wedge D_2 \wedge \text{Expl})$ can be transformed into $(Q_2 \wedge \text{Expl})$ by performing an unfolding step, and by Step 3 program Q_1 can be obtained from Q_2 by performing a folding step.

The preceding implicit definition strategy generalizes the one presented in Kanamori and Maeji [1986] and it is related to the technique described in Kott [1982] for proving properties of functional programs based on unfold/fold transformations.

3.3 Examples of Transformations of Logic Programs

In Example 3.3.1 we show that the rules and strategies previously described can be used for reducing the amount of nondeterminism in a given program. In particular, we make use of the tupling, loop absorption, and implicit definition strategies. We end this section by demonstrating in Example 3.3.2 the use of the generalization strategy.

Example 3.3.1 (Prime Numbers). Let us consider the problem of computing the list P of the first N prime numbers for any given natural number $N > 0$. To this purpose we define a relation *primes*(N, P) by means of the following program, called *Primes*:

$$\text{primes}(N, P) \leftarrow \text{initial}(L), \text{not_div_by_smaller}(L, P), \text{length}(P, N) \quad (66)$$

$$\text{initial}([2]) \leftarrow \quad (67)$$

$$\text{initial}([H, K|T]) \leftarrow \text{initial}([K|T]), \text{plus}(K, 1, H) \quad (68)$$

$$\text{not_div_by_smaller}([], []) \leftarrow \quad (69)$$

$$\text{not_div_by_smaller}([H|T], P) \leftarrow \text{not_div_by_smaller}(T, Q), \text{div_tests}(H, Q, P) \quad (70)$$

$$\text{div_tests}(H, Q, [H|Q]) \leftarrow \text{not_div_any}(H, Q) \quad (71)$$

$$\text{div_tests}(H, Q, Q) \leftarrow \text{div_some}(H, Q) \quad (72)$$

$$\text{length}([], 0) \leftarrow \quad (73)$$

$$\text{length}([H|T], N) \leftarrow \text{length}(T, N1), \text{plus}(N1, 1, N) \quad (74)$$

where we assume that: $\text{plus}(X, Y, Z)$ holds iff $X + Y = Z$; $\text{not_div_any}(H, L)$ holds iff H is not divisible by any number of the list L ; and $\text{div_some}(H, L)$ holds iff H is divisible by a number in L .

Notice that: $\text{initial}(L)$ holds iff L is the list $[M, M-1, \dots, 2]$ of consecutive natural numbers for some number $M(\geq 2)$; $\text{not_div_by_smaller}(L, P)$ holds iff P is the sublist of L such that an element of L , say X , is in P iff for each Y which is to the right of X in L , X is not divisible by Y ; and $\text{length}(P, N)$ holds iff the length of the list P is N .

Let us assume the left-to-right, depth-first Prolog evaluation strategy and suppose that the program *Primes* is evaluated for a goal of the form $\text{primes}(N, P)$, where N is bound to a positive number and P is an unbound variable. The execution corresponding to a goal of that form has a very large degree of nondeterminism. Indeed, the program *Primes* generates an initial segment, say L , of numbers greater than 1, then it constructs the list P of the prime numbers contained in L , and it finally matches the length of P with the given N . If the matching fails, the program *Primes* generates the next initial segment of natural numbers and it constructs the prime numbers contained in that segment without taking advantage of the results of the divisibility tests performed by div_tests before failure. If we assume that the evaluation of $\text{div_tests}(H, Q, R)$ performs $O(\text{length}(Q))$ tests of divisibility, then the program *Primes* performs $O(K \times N^2)$ tests of divisibility, where K is the largest generated prime number and N is the number of generated primes (i.e., N is the length of P).

In the following derivation we improve the program *Primes* by avoiding the construction of the intermediate lists which are shared by the predicates *initial*, *not_div_by_smaller*, and *length*, and reducing the nondeterminism of the derived program.

For clarity, we break our derivation into the five parts.

Part 1. Application of the tupling and loop absorption strategies for avoiding intermediate lists. We begin our transformation process by applying the tupling strategy to the atoms $\text{initial}(L)$, $\text{not_div_by_smaller}(L, P)$, and $\text{length}(P, N)$ which share the variables L and P in the body of Clause (66). Because the atoms to be tupled together constitute the whole body of Clause (66) defining the predicate *primes*, we do not need to introduce (by the definition rule) a new predicate and we look for the recursive definition of the predicate *primes*. By unfolding, from Clause (66) we get:

$$\text{primes}(1, [2]) \leftarrow \quad (75)$$

$$\text{primes}(N, [H|P]) \leftarrow \text{initial}([K|T]), \text{plus}(K, 1, H), \text{not_div_by_smaller}([K|T], P), \quad (76)$$

$$\text{not_div_any}(H, P), \text{length}(P, N1), \text{plus}(N1, 1, N)$$

$$\text{primes}(N, P) \leftarrow \text{initial}([K|T]), \text{plus}(K, 1, H), \text{not_div_by_smaller}([K|T], P), \quad (77)$$

$$\text{div_some}(H, P), \text{length}(P, N).$$

The bodies of Clauses (76) and (77) both contain instances of the body of Clause (66). However, we cannot perform any folding step using Clause (66) because the variable L of Clause (66) does not occur in the head. Thus we are in a situation

where we may apply the loop absorption strategy. By doing so, we introduce the new clause:

$$new1(L, N, P) \leftarrow initial(L), not_div_by_smaller(L, P), length(P, N) \quad (78)$$

whose body is equal to that of Clause (66), and whose head contains the variables required for folding.

By performing a few unfolding and folding steps we get the following recursive definition of *new1*:

$$new1([2], [2], 1) \leftarrow \quad (79)$$

$$new1([H, K|T], [H|P], N) \leftarrow new1([K|T], P, N1), plus(K, 1, H), \quad (80)$$

$$not_div_any(H, P), plus(N1, 1, N)$$

$$new1([H, K|T], P, N) \leftarrow new1([K|T], P, N), plus(K, 1, H), div_some(H, P). \quad (81)$$

By folding we obtain a definition of the predicate *primes* in terms of the new predicate *new1*:

$$primes(N, P) \leftarrow new1(L, P, N). \quad (82)$$

Part 2. Fusion of clauses with common atoms. Now we notice that the predicate *new1* occurs in the body of both Clause (80) and Clause (81). This fact may cause the repeated evaluation of identical atoms when backtracking occurs. We can avoid this repeated evaluation if we replace Clauses (80) and (81) by the following three clauses:

$$new1([H, K|T], P, N) \leftarrow new1([K|T], P1, N1), p(K, H, P, N, P1, N1) \quad (83)$$

$$p(H, K, [H|P], N, P, N1) \leftarrow plus(K, 1, H), not_div_any(H, P), plus(N1, 1, N) \quad (84)$$

$$p(H, K, P, N, P, N) \leftarrow plus(K, 1, H), div_some(H, P). \quad (85)$$

This transformation is sometimes called *clause fusion* [Debray and Warren 1988; Deville 1990] and its correctness is ensured by the fact that Clauses (80) and (81) can be obtained by unfolding *p* in Clause (83) using Clauses (84) and (85).

The derived program consisting of Clauses (82), (79), (83), (84), and (85), is more efficient than the initial program because it avoids the visit of some lists. In particular, the atom *new1(L, P, N)*, while generating a segment *L* of integers, collects in the list *P* the prime numbers occurring in *L* and also computes the length *N* of *P*. On the other hand, the initial program performs the same computations by using distinct predicates that first construct and then visit the lists *L* and *P*.

Part 3. Use of the implicit definition strategy for reducing nondeterminism. In order to allow a clearer presentation of the transformations that we indicate in the following, we begin by rewriting the program we have derived so far, as follows:

$$primes(N, P) \leftarrow new1(L, P, N) \quad (82)$$

$$new1(L, P, N) \leftarrow L = [2], P = [2], N = 1 \quad (86)$$

$$new1(L, P, N) \leftarrow new1(L1, P1, N1), q(L, P, N, L1, P1, N1) \quad (83^*)$$

$$q([H, K|T], [H|P1], N, [K|T], P1, N1) \leftarrow \quad (84^*)$$

$$plus(K, 1, H), not_div_any(H, P1), plus(N1, 1, N)$$

$$q([H, K|T], P1, N, [K|T], P1, N) \leftarrow plus(K, 1, H), div_some(H, P1) \quad (85^*)$$

The correctness of this rewriting follows from the fact that by unfolding the equalities in Clause (86) we get Clause (79) and by unfolding q in Clause (83*) we get Clauses (80) and (81) which are equivalent to Clauses (83)–(85).

The main cause of nondeterminism in the preceding program is Clause (83*), where backtracking steps may be generated during the evaluation of $q(L, P, N, L1, P1, N1)$ after the evaluation of the recursive call of *new1*. In order to reduce nondeterminism we want to anticipate the computation of q , so that only nonfailing recursive calls to *new1* are made. This goal can be achieved by obtaining a tail recursive version of Clause (83*). In other words, because we are assuming a left-to-right evaluation order of the atoms in a goal, we would like to get a clause equivalent to (83*) in which the recursive call to *new1* is the rightmost one.

The reader should notice that, in general, we cannot get an efficient tail recursive Prolog program by simply rearranging the order of the atoms in the body of the clauses. Indeed, in the case of Clause (83*) by moving the atom $q(L, P, N, L1, P1, N1)$ to the left of the atom *new1*($L1, P1, N1$) we get a tail recursive clause, but the nondeterminism of the program is increased, at least for the class of goals we consider. The formal study of program properties that depend on the form of the goals, such as nondeterminism, can be done by using abstract interpretations (see, for instance, Jones and Søndergaard [1987]).

We now transform Clause (83*) into a tail recursive clause of the following form:

$$new1(L, P, N) \leftarrow B[L, P, N, \bar{X}], New2[L, P, N, \bar{X}] \quad (87)$$

the notation $G[\cdot \cdot \cdot]$ is used here and in the sequel for denoting that a given goal (or atom) G depends on some of the variables occurring in the list $[\cdot \cdot \cdot]$; B is a goal made out of atoms with predicate symbols q and “=”; $New2$ is an atom with a possibly new predicate symbol; and $\bar{X}$ is an n -tuple of variables.

We now apply the implicit definition strategy to compute the goal B and the explicit definition, say *Expl2*, for the atom *New2*. According to that strategy, we perform the following steps:

Step 1. We consider a new predicate *new3* and two different definition clauses for it:

$$D_1. \quad new3(L, P, N) \leftarrow new1(L1, P1, N1), q(L, P, N, L1, P1, N1),$$

whose body is identical to that of Clause (83*); and

$$D_2. \quad new3(L, P, N) \leftarrow B[L, P, N, \bar{X}], New2[L, P, N, \bar{X}],$$

whose body is identical to that of Clause (87).

Step 2. By unfolding *new1* in clause D_1 and then folding using clause D_1 itself, we get the following recursive definition of *new3*:

$$new3(L, P, N) \leftarrow L1 = [2], P1 = [2], N1 = 1, q(L, P, N, L1, P1, N1) \quad (88)$$

$$new3(L, P, N) \leftarrow new3(L2, P2, N2), q(L, P, N, L2, P2, N2). \quad (89)$$

Step 3. We then unfold *new3* in Clause (89) by using clause D_2 . Thus we replace

Clause (89) by the following one:

$$new3(L, P, N) \leftarrow B[L2, P2, N2, \bar{X}], New2[L2, P2, N2, \bar{X}], q(L, P, N, L2, P2, N2). \quad (90)$$

Step 4. We now assume that Clauses (88) and (90) are obtained by unfolding only the atom *New2* in Clause D_2 using the set *Expl2* of unknown clauses. By this assumption we are able to determine the goal *B* and the set *Expl2* of clauses that explicitly define the atom *New2*, as we now indicate.

Because we assume that $B[L, P, N, \bar{X}]$ is not unfolded, an occurrence of it should be in the body of Clause (88). Among many possible choices we may take $B[L, P, N, \bar{X}]$ to be the goal $(L1=[2], P1=[2], N1=1)$, thereby instantiating $\bar{X}$ to the triplet of variables $\langle L1, P1, N1 \rangle$. Thus Clauses D_2 and (90) can be rewritten as follows:

$$new3(L, P, N) \leftarrow L1 = [2], P1 = [2], N1 = 1, New2[L, P, N, L1, P1, N1] \quad (91)$$

$$new3(L, P, N) \leftarrow L1 = [2], P1 = [2], N1 = 1, \quad (92)$$

$$New2[L2, P2, N2, L1, P1, N1], q(L, P, N, L2, P2, N2).$$

According to our hypotheses, by unfolding *New2* in Clause (91) we should obtain Clauses (88) and (92). Therefore the clauses that define the atom *New2* with the new predicate symbol *new2* are:

$$new2(L, P, N, L1, P1, N1) \leftarrow q(L, P, N, L1, P1, N1) \quad (93)$$

$$new2(L, P, N, L1, P1, N1) \leftarrow new2(L2, P2, N2, L1, P1, N1), \quad q(L, P, N, L2, P2, N2). \quad (94)$$

Clauses (93) and (94) constitute the set *Expl2* which we wanted to compute.

As a final step of the application of the implicit definition strategy, we replace Clause (83*) by the following tail recursive clause:

$$new1(L, P, N) \leftarrow L1 = [2], P1 = [2], N1 = 1, new2(L, P, N, L1, P1, N1). \quad (95)$$

Thus the version of the program *Primes* we have derived so far is made out of the following clauses:

$$primes(N, P) \leftarrow new1(L, P, N) \quad (82)$$

$$new1(L, P, N) \leftarrow L = [2], P = [2], N = 1 \quad (86)$$

$$new1(L, P, N) \leftarrow L1 = [2], P1 = [2], N1 = 1, new2(L, P, N, L1, P1, N1) \quad (95)$$

$$new2(L, P, N, L1, P1, N1) \leftarrow q(L, P, N, L1, P1, N1) \quad (93)$$

$$new2(L, P, N, L1, P1, N1) \leftarrow new2(L2, P2, N2, L1, P1, N1), q(L, P, N, L2, P2, N2) \quad (94)$$

together with the clauses for *q*, *not_div_any*, *div_some*, and *plus*.

Part 4. Further use of the implicit definition strategy for reducing nondeterminism. Unfortunately, Clause (94) introduced during the transformation of Clause

(83*) into tail recursive form, is *not* tail recursive. Thus we again apply the implicit definition strategy for transforming Clause (94) into one of the form:

$new2(L, P, N, L1, P1, N1) \leftarrow$

$$C[L, P, N, L1, P1, N1, \bar{Y}], New4[L, P, N, L1, P1, N1, \bar{Y}], \quad (96)$$

where C is a conjunction of atoms with predicates q and “=”, and $New4$ is an atom with a possibly new predicate symbol. In order to determine the form of the goal C and the explicit definition of the atom $New4$, we consider a new predicate $new5$ and two different definitions for it, corresponding to the bodies of Clauses (94) and (96), respectively:

$E_1. \quad new5(L, P, N, L1, P1, N1) \leftarrow$

$$new2(L2, P2, N2, L1, P1, N1), q(L, P, N, L2, P2, N2);$$

$E_2. \quad new5(L, P, N, L1, P1, N1) \leftarrow$

$$C[L, P, N, L1, P1, N1, \bar{Y}], New4[L, P, N, L1, P1, N1, \bar{Y}].$$

A derivation similar to the derivation shown previously when constructing the explicit definition $Expl2$ of $new2$, allows us to rewrite Clause E_2 as follows:

$new5(L, P, N, L1, P1, N1) \leftarrow$

$$q(L2, P2, N2, L1, P1, N1), new4(L, P, N, L2, P2, N2) \quad (97)$$

with the following explicit definition of $new4$:

$$new4(L, P, N, L1, P1, N1) \leftarrow q(L, P, N, L1, P1, N1) \quad (98)$$

$new4(L, P, N, L1, P1, N1) \leftarrow$

$$new4(L2, P2, N2, L1, P1, N1), q(L, P, N, L2, P2, N2). \quad (99)$$

The clauses for $new4$ are identical (up to the name of the predicate) to the clauses for $new2$. Therefore we may forget about Clauses (98) and (99), and we may replace the body of Clause (94) by the body of Clause (97) with $new2$ written instead of $new4$. Thus instead of Clause (94), we get the following tail recursive clause:

$new2(L, P, N, L1, P1, N1) \leftarrow$

$$q(L2, P2, N2, L1, P1, N1), new2(L, P, N, L2, P2, N2). \quad (100)$$

The current program version is tail recursive and it is made out of the following clauses:

$$primes(N, P) \leftarrow new1(L, P, N) \quad (82)$$

$$new1(L, P, N) \leftarrow L = [2], P = [2], N = 1 \quad (86)$$

$$new1(L, P, N) \leftarrow L1 = [2], P1 = [2], N1 = 1, new2(L, P, N, L1, P1, N1) \quad (95)$$

$$new2(L, P, N, L1, P1, N1) \leftarrow q(L, P, N, L1, P1, N1) \quad (93)$$

$new2(L, P, N, L1, P1, N1) \leftarrow$

$$q(L2, P2, N2, L1, P1, N1), new2(L, P, N, L2, P2, N2) \quad (100)$$

together with the clauses for q , not_div_any , div_some , and $plus$.

A straightforward transformation, consisting of the fusion of Clauses (93) and (100), and the unfolding of the predicates $new1$, $new2$, q , and “=” allows us to get the following program:

$$primes(1, [2]) \leftarrow \quad (75)$$

$$primes(N, P) \leftarrow new6(L, P, N, [3, 2], [3, 2], 2) \quad (101)$$

$$new6(L, P, N, L, P, N) \leftarrow \quad (102)$$

$$new6(L, P, N, [K|T], P1, N1) \leftarrow plus(K, 1, H), plus(N1, 1, N2), \quad (103)$$

$$not_div_any(H, P1),$$

$$new6(L, P, N, [H, K|T], [H|P1], N2)$$

$$new6(L, P, N, [K|T], P1, N1) \leftarrow plus(K, 1, H), div_some(H, P1), \quad (104)$$

$$new6(L, P, N, [H, K|T], P1, N1)$$

Part 5. Elimination of unnecessary variables by applying the tupling strategy. The binding computed for the variable L in Clause (101) is not explicitly used for describing the relation $primes(N, P)$, because L does not occur in the head. To avoid the occurrence of that variable, we apply the tupling strategy for $n=1$; that is, we tuple one atom only. Thus we introduce a new predicate $new7$ defined as follows:

$$new7(P, N, L1, P1, N1) \leftarrow new6(L, P, N, L1, P1, N1) \quad (105)$$

By folding Clause (101) using Clause (105) and then deriving the recursive definition of $new7$ we eliminate the unnecessary variable L . We get the following final program:

$$primes(1, [2]) \leftarrow \quad (75)$$

$$primes(N, P) \leftarrow new7(P, N, [3, 2], [3, 2], 2) \quad (106)$$

$$new7(P, N, L, P, N) \leftarrow \quad (107)$$

$$new7(P, N, [K|T], P1, N1) \leftarrow$$

$$plus(K, 1, H), not_div_any(H, P1), plus(N1, 1, N2), \quad (108)$$

$$new7(P, N, [H, K|T], [H|P1], N2)$$

$$new7(P, N, [K|T], P1, N1) \leftarrow plus(K, 1, H), div_some(H, P1), \quad (109)$$

$$new7(P, N, [H, K|T], P1, N1).$$

This program keeps a list $P1$ of the prime numbers generated so far together with its length $N1$ (see the fourth and fifth arguments of $new7$, respectively), and it terminates when $N1$ reaches the desired value N (see Clause (107)). The list $P1$ of prime numbers is updated by considering a number at a time and testing its divisibility by the primes generated so far.

Thus this final program performs $O(K \times N)$ divisibility tests, where K is the largest generated prime number and N is the number of generated primes.

The following final example shows the use of the generalization strategy.

Example 3.3.2 (Minimal Leaf Replacement). Suppose that we are given a binary tree, say $InTree$, whose leaves are labeled by numbers. We want to obtain another tree, say $OutTree$, of the same shape with all its leaves replaced by their minimal value.

A simple two-visit algorithm can be given as follows. We first compute the minimal leaf value, say Min , of $InTree$, and then we visit again that tree for replacing its

leaves by *Min*. A logic program that realizes this algorithm is as follows:

$$\begin{aligned} \text{mintree}(\text{InTree}, \text{OutTree}) \leftarrow & \text{minleaves}(\text{InTree}, \text{Min}), & (110) \\ & \text{replace}(\text{Min}, \text{InTree}, \text{OutTree}) \end{aligned}$$

$$\text{minleaves}(\text{tip}(N), N) \leftarrow \quad (111)$$

$$\begin{aligned} \text{minleaves}(\text{tree}(L, R), \text{Min}) \leftarrow & \text{minleaves}(L, \text{MinL}), \text{minleaves}(R, \text{MinR}), & (112) \\ & \text{min}(\text{MinL}, \text{MinR}, \text{Min}) \end{aligned}$$

$$\text{replace}(M, \text{tip}(N), \text{tip}(M)) \leftarrow \quad (113)$$

$$\begin{aligned} \text{replace}(\text{Min}, \text{tree}(\text{InL}, \text{InR}), \text{tree}(\text{OutL}, \text{OutR})) \leftarrow & \text{replace}(\text{Min}, \text{InL}, \text{OutL}), & (114) \\ & \text{replace}(\text{Min}, \text{InR}, \text{OutR}) \end{aligned}$$

where $\text{min}(M1, M2, M)$ holds iff M is the minimum number of $M1$ and $M2$.

We would like to derive a program that traverses *InTree* only once. To this aim we may apply the tupling strategy to the predicates *minleaves* and *replace* which share the argument *InTree*. As in the first application of the tupling strategy in Example 3.3.1, a new definition corresponding to the body of Clause (110) is not necessary. We only need to look for the recursive definition of the predicate *mintree*. After some unfolding steps, we get:

$$\text{mintree}(\text{tip}(N), \text{tip}(N)) \leftarrow \quad (115)$$

$$\begin{aligned} \text{mintree}(\text{tree}(\text{InL}, \text{InR}), \text{tree}(\text{OutL}, \text{OutR})) \leftarrow \\ \text{minleaves}(\text{InL}, \text{MinL}), \text{minleaves}(\text{InR}, \text{MinR}), & (116) \\ \text{min}(\text{MinL}, \text{MinR}, \text{Min}), \\ \text{replace}(\text{Min}, \text{InL}, \text{OutL}), \text{replace}(\text{Min}, \text{InR}, \text{OutR}). \end{aligned}$$

Now, looking for a fold of the goal “*minleaves*(*InL*, *MinL*), *replace* (*Min*, *InL*, *OutL*)” using Clause (110), we realize that no matching is possible because this goal is not an instance of “*minleaves*(*InTree*, *Min*), *replace*(*Min*, *InTree*, *OutTree*).” Thus we apply the generalization strategy and we introduce the following clause:

$$\begin{aligned} G. \text{ genmintree}(\text{InTree}, M1, M2, \text{OutTree}) \leftarrow & \text{minleaves}(\text{InTree}, M1), \\ & \text{replace}(M2, \text{InTree}, \text{OutTree}), \end{aligned}$$

where $\text{bd}(G)$ is the most concrete generalization of the two goals to be folded, that is, “*minleaves*(*InL*, *MinL*), *replace*(*Min*, *InL*, *OutL*)” and the body of Clause (110). By folding Clause (110) we get:

$$\text{mintree}(\text{InTree}, \text{OutTree}) \leftarrow \text{genmintree}(\text{InTree}, \text{Min}, \text{Min}, \text{OutTree}). \quad (117)$$

We are now left with the problem of finding the recursive definition of the predicate *genmintree*. This is an easy task, because we can perform the unfolding steps corresponding to those leading from Clause (110) to Clauses (115) and (116), and then we can use Clause *G* for folding. After those steps we get the following final program, which performs the desired tree transformation in one visit.

$$\text{mintree}(\text{InTree}, \text{OutTree}) \leftarrow \text{genmintree}(\text{InTree}, \text{Min}, \text{Min}, \text{OutTree}) \quad (117)$$

$$\text{genmintree}(\text{tip}(N), N, M, \text{tip}(M)) \leftarrow \quad (118)$$

$$\begin{array}{l}
\text{genmintree}(\text{tree}(\text{InL}, \text{InR}), M1, M2, \text{tree}(\text{OutL}, \text{OutR})) \leftarrow \\
\qquad \text{genmintree}(\text{InL}, M1L, M2, \text{OutL}), \\
\qquad \text{genmintree}(\text{InR}, M1R, M2, \text{OutR}), \\
\qquad \text{min}(M1L, M1R, M1).
\end{array} \tag{119}$$

Let us now consider an execution of the derived program for evaluating a goal of the form “*mintree*(*t*, *T*),” where *t* is a ground binary tree and *T* is an unbound variable. During the visit of the input tree *t*, the predicate *genmintree* both computes the minimal leaf value *M1* and replaces the leaves using the *unbound* variable *M2*. The instantiation of *M2* to the minimal leaf value is performed by the unification of the variables *M1* and *M2* due to the first clause of our final program.

When writing a similar program in an imperative language, in order to realize the leaf replacement in one visit we should use pointers to the location that at the end holds the minimal leaf value. As already seen in Example 2.3.6, the derivation of programs that use pointers in a disciplined way can be done by applying the Lambda Abstraction strategy.

4. OTHER TECHNIQUES FOR TRANSFORMING FUNCTIONAL AND LOGIC PROGRAMS

It is difficult to exhaustively report on the various techniques and methodologies proposed in the literature for deriving functional or logic programs by transformation. As an introduction the reader may refer to Feather [1987], Partsch [1990], Möller et al. [1993], and Pettorossi and Proietti [1994].

We present the following techniques:

- (1) the *Schemata Approach* (one may refer, for instance, to Partsch [1990] for a survey in the functional case, and to Brough and Hogger [1991], Deville and Burnay [1989], Fuchs and Fromherz [1992], Marakakis and Gallagher [1994], and Seki and Furukawa [1987] in the logic case),
- (2) the *Partial Evaluation* (or *Mixed Computation*) technique [Ershov 1982; Jones et al. 1993], and
- (3) the *Finite Differencing* technique [Paige and Koenig 1982], and compare them with the strategies we have previously considered.

For other methods, such as: combinatory techniques [Turner 1979], supercombinator methods [Hughes 1982], local recursions [Bird 1984a], promotion strategies [Bird 1984b], lambda liftings [Johnsson 1985], and lambda hoistings [Takeichi 1987], we suggest a look at the original papers.

There are some other techniques that have been proposed in the literature for transforming functional and logic programs. They are related to higher order features, in the sense that *continuations* and *communications* among function calls and predicate evaluations are created for the derivation of very efficient programs. The reader may refer to Wand [1980] and Pettorossi and Skowron [1982] for the functional case and to Tarau and Boyer [1990] for the logic case.

For an introductory view of some methods for the transformation and derivation of parallel and/or concurrent programs the reader may refer to Paige et al. [1993].

4.1 Schemata Approach

In the *schemata approach* to program transformation the programmer is given a *dictionary*, that is, a set of pairs of program schemata. If a program *P* matches a schema *S*₁ by a substitution *θ* and *⟨S*₁, *S*₂*⟩* is a pair in the dictionary, we can

replace P by $S_2 \theta$. The dictionary ensures that P and $S_2 \theta$ are semantically equivalent, and also that $S_2 \theta$ is an improvement over P w.r.t. some complexity measure.

We presented some instances of these schemata transformations in the case of functional languages in Examples 2.3.2, 2.3.3, 2.3.5 and 2.3.6 (see Figures 2, 4, 5, and 6, respectively). Similar transformations can also be applied in the case of logic languages. Moreover, in Seki and Furukawa [1987] it is shown that by schemata transformations one can reduce the nondeterminism of generate-and-test programs.

An advantage of the schemata approach over the “rules + strategy” approach is that the application of a schema transformation is not computationally expensive, because it only requires the application of a substitution. However, the choice of a suitable schema transformation in the given dictionary of the available transformations may be time consuming, because it also requires looking for the existence of a matching substitution.

A drawback of the schemata approach is the impossibility of performing any kind of transformation when the program at hand is not an instance of any schema in the dictionary.

As an example of the schemata approach we want to mention the *inversion of the flow of computation* [Boiten 1990]. This technique can be applied when the initial program is defined in terms of operators that have inverses or satisfy some suitable properties, such as associativity or commutativity. By applying this technique, repeated evaluations of functions are often avoided and efficiency is improved. For instance, by inverting the flow of computation of the initial program:

$fib(n) =$

```

if  $n \leq 1$  then 1
      else  $fib(n - 1) + fib(n - 2)$ 
for  $n \geq 0$ ,

```

which computes the Fibonacci function in exponential time, we get the linear time program:

$fib(n) = D(0, n, 1, 1)$ **for** $n \geq 0$

$D(m, n, a, b) =$

```

if  $m = n$  then  $b$ 
      else  $D(m + 1, n, a + b, a)$ 
for  $m, n \geq 0$ .

```

The flow of computation has been inverted in the sense that the computation of the initial program is top-down, whereas the computation of the final program is bottom-up.

The preceding transformation can be performed in one step only, by applying a suitable schema equivalence, after having checked the validity of the required conditions on the operators [Boiten 1990]. The correctness of that transformation is based on the fact that the successor function is the inverse of the predecessor function and for $n \geq 0$ the unfolding of $fib(n)$ eventually generates the calls of $fib(0)$ and/or $fib(1)$ only.

Obviously, in many cases, as in this Fibonacci example, instead of applying a schema equivalence, one can derive the final program via the application of the transformation rules listed in Section 2.

4.2 Partial Evaluation

Partial Evaluation (related to *Program Division* [Jones 1987] and *Variable Splitting* [Bjørner et al. 1988]) is defined via a function $PEval: Programs \times Data_1 \rightarrow Programs$, which given a program P in *Programs* and a data d_1 in the domain $Data_1$, produces a new program, called the *residual program*, as we now specify. Suppose that the domain of P is isomorphic to $Data_1 \times Data_2$, where $Data_2$ is a suitable domain. Then for every data d_2 in $Data_2$ the residual program $PEval(P, d_1)$ of P w.r.t. d_1 is a program that satisfies the following equation:

$$P\langle d_1, d_2 \rangle = (PEval(P, d_1))d_2.$$

Partial evaluation is used for improving program performance when d_1 , called the *static* data, is known at compile-time. The residual program is obtained by “importing” the information relative to the static data d_1 into the program P . The data d_2 is known only at run-time and is called the *dynamic* data.

Partial evaluation is also used for automatically producing compilers from interpreters [Futamura 1971].

Partial evaluation can be viewed as the inverse of Lambda Abstraction in the sense that the latter is defined by the function $Lambda: Programs \rightarrow (Programs \times Data)$, which “exports” information from a given program and, for any program P and data d , satisfies the following equation [Pettorossi and Proietti 1987]:

$$P d = P_1\langle d_1, d \rangle$$

$$\text{where } Lambda(P) = \langle P_1, d_1 \rangle.$$

The partial evaluation technique has also been widely applied in the logic programming context, where it was first studied in Komorowski [1982]. In this context, the partial evaluation (also called *partial deduction*) of a program P w.r.t. a goal G produces a program P_G such that, every ground goal $G\theta$ is true in $M(P)$ iff $G\theta$ is true in $M(P_G)$.

The basic theoretical results about the correctness of partial evaluation of logic programs, as well as the extension of this technique to the case of general logic programs with negation, can be found in Lloyd and Shepherdson [1991]. For a tutorial introduction to partial evaluation of logic programs the reader may look at Gallagher [1993].

In the following example we illustrate the use of the partial evaluation technique for the derivation of an efficient acceptor for a regular expression given at compile-time, starting from a “universal acceptor” which works for any regular expression given at run-time.

In the example we use the transformation rules and strategies presented in Section 3, instead of ad hoc techniques for partial evaluation such as those considered in Lloyd and Shepherdson [1991] and Gallagher [1993].

Example 4.2.1 (Acceptors for Regular Expressions). Let us consider the following logic program, called *Accepts*, for testing whether a word W in $\{a, b\}^*$ is accepted by the regular expression R [Mogensen and Bondorf 1993]. This test is performed by a goal of the form *accepts*(R, W), where the word W is represented as a

(possibly empty) list of as and bs , and R is a term denoting a regular expression in the set Reg defined as follows:

$$Reg ::= void \mid a \mid b \mid (Reg; Reg) \mid (Reg + Reg) \mid Reg^*,$$

where $void$ is the regular expression which does not accept any word.

```

accepts( $R$ , [ ])  $\leftarrow$  accepts_empty( $R$ )
accepts( $R$ , [ $S|W$ ])  $\leftarrow$  next( $R$ ,  $S$ ,  $R1$ ), accepts( $R1$ ,  $W$ )
accepts_empty(( $R1$ ;  $R2$ ))  $\leftarrow$  accepts_empty( $R1$ ), accepts_empty( $R2$ )
accepts_empty(( $R1 + R2$ ))  $\leftarrow$  accepts_empty( $R1$ )
accepts_empty(( $R1 + R2$ ))  $\leftarrow$  accepts_empty( $R2$ )
accepts_empty( $R^*$ )  $\leftarrow$ 
next( $R$ ,  $S$ ,  $R1$ )  $\leftarrow$  transf( $R$ ,  $TR$ ), deriv( $TR$ ,  $S$ ,  $L$ ), make_regexpr( $L$ ,  $R1$ )
transf( $void$ ,  $void$ )  $\leftarrow$ 
transf( $S$ ,  $S$ )  $\leftarrow$  symbol( $S$ )
transf(( $void$ ;  $R$ ),  $void$ )  $\leftarrow$ 
transf(( $S$ ;  $R$ ), ( $S$ ;  $R$ ))  $\leftarrow$  symbol( $S$ )
transf((( $R1$ ;  $R2$ );  $R3$ ),  $R4$ )  $\leftarrow$  transf(( $R1$ ;  $R2$ ;  $R3$ )),  $R4$ )
transf((( $R1 + R2$ );  $R3$ ),  $R4$ )  $\leftarrow$  transf((( $R1$ ;  $R3$ ) +  $R2$ ;  $R3$ )),  $R4$ )
transf(( $R1^*$ ;  $R2$ ),  $R3$ )  $\leftarrow$  transf(( $R2 + (R1; (R1^*; R2))$ ),  $R3$ )
transf(( $R1 + R2$ ), ( $R3 + R4$ ))  $\leftarrow$  transf( $R1$ ,  $R3$ ), transf( $R2$ ,  $R4$ )
transf( $R1^*$ , ( $void^* + R2$ ))  $\leftarrow$  transf(( $R1$ ;  $R1^*$ ),  $R2$ )
deriv( $void$ ,  $S$ , [ ])  $\leftarrow$ 
deriv( $void^*$ ,  $S$ , [ ])  $\leftarrow$ 
deriv( $S$ ,  $S$ , [ $void^*$ ])  $\leftarrow$  symbol( $S$ )
deriv( $S1$ ,  $S2$ , [ ])  $\leftarrow$  symbol( $S1$ ), symbol( $S2$ ),  $S1 \neq S2$ 
deriv(( $S$ ;  $R$ ),  $S$ , [ $R$ ])  $\leftarrow$  symbol( $S$ )
deriv(( $S1$ ;  $R$ ),  $S2$ , [ ])  $\leftarrow$  symbol( $S1$ ), symbol( $S2$ ),  $S1 \neq S2$ 
deriv(( $R1 + R2$ ),  $S$ ,  $L$ )  $\leftarrow$  deriv( $R1$ ,  $S$ ,  $L1$ ), deriv( $R2$ ,  $S$ ,  $L2$ ), append( $L1$ ,  $L2$ ,  $L$ )
make_regexpr([ $R$ ],  $R$ )  $\leftarrow$ 
make_regexpr([ $R1$ ,  $R2|Rs$ ], ( $R1 + T$ ))  $\leftarrow$  make_regexpr([ $R2|Rs$ ],  $T$ )
symbol( $a$ )  $\leftarrow$ 
symbol( $b$ )  $\leftarrow$ 

```

where:

- $accepts_empty(R)$ holds iff the regular expression R accepts the empty word [];
- $next(R, S, R1)$ holds iff S is a symbol in $\{a, b\}$ and for every word W in $\{a, b\}^*$, W is accepted by the regular expression R iff there exists a word Z accepted by the regular expression $R1$ such that $W = S; Z$. Thus $R1$ is the *derivative* of R w.r.t. S [Brzozowski 1964]. $R1$ is computed as the following steps specify.

- Step 1.* First the predicate $transf(R, TR)$ transforms the regular expression R into an equivalent regular expression TR in the set $TReg$ defined as follows: $TReg ::= void|void^*|a|b|(a; Reg)|(b; Reg)|(TReg + TReg)$, where $void^*$ is the regular expression that accepts the empty word only. This transformation makes it very simple to compute the leftmost symbol of any word accepted by a given regular expression R .
- Step 2.* Then the predicate $deriv(TR, S, L)$ computes the list L of all regular expressions U such that $(S; U)$ is a *summand* of TR , where, given any regular expression $E = E_1 + \dots + E_n$, for $n \geq 1$ and any possible parenthesization of $E_1 + \dots + E_n$, a summand of E is E_i for any $i = 1, \dots, n$. The predicate $deriv(TR, S, L)$ also includes the regular expression $void^*$ in L iff S is a summand of TR .
- Step 3.* Finally, the predicate $make_regexpr(L, R1)$ transforms the list L made out of the regular expressions $E_1, \dots, E_{n-1}, E_n$ into the regular expression $R1$ which is $(E_1 + (\dots + (E_{n-1} + E_n) \dots))$.

For reasons of simplicity, during partial evaluation, we also write terms without their outermost parentheses, we assume the right associativity of “;” and “+”, and we assume that “*” binds more than “;” and “;” binds more than “+”. Thus, for instance, the term $(R1; (R2; R3))$ may also be written as $R1; R2; R3$.

From the preceding program *Accepts*, we would like to derive a residual program for testing whether a given word W in $\{a, b\}^*$ is accepted by the regular expression $(a+b)^*; a; b; a$. This can be done by partially evaluating the program w.r.t. the goal $accepts(((a+b)^*; a; b; a), W)$.

We start off by introducing, via the definition rule, a new clause whose body is the goal w.r.t. that we want to perform the partial evaluation. Thus, in our case, we introduce the following clause:

$$new0(W) \leftarrow accepts(((a+b)^*; a; b; a), W). \quad (120)$$

We then look for a set of clauses that define the predicate *new0*, and whose bodies do not contain any instantiated atom. Indeed, if there are instantiated atoms we may continue the partial evaluation process for taking full advantage of the information available. By repeatedly applying the unfolding rule and the clause deletion rule, we get from Clause (120) the following clauses:

$$new0([a|W]) \leftarrow accepts((b; a + (a+b)^*; a; b; a), W) \quad (121)$$

$$new0([b|W]) \leftarrow accepts(((a+b)^*; a; b; a), W). \quad (122)$$

Clause (122) can be folded using Clause (120), thereby getting:

$$new0([b|W]) \leftarrow new0(W), \quad (123)$$

whose body does not contain any instantiated atom.

We now continue the derivation from Clause (121). After a few unfolding steps and clause deletion steps we get:

$$new0([a, a|W]) \leftarrow accepts((b; a + (a+b)^*; a; b; a), W) \quad (124)$$

$$new0([a, b|W]) \leftarrow accepts((a + (a+b)^*; a; b; a), W). \quad (125)$$

Because the body of Clause (124) is a variant of the body of Clause (121) we may apply the loop absorption strategy. Thus we introduce a new predicate, called *new1*, defined by the following clause whose body is equal to the body of Clause (121):

$$new1(W) \leftarrow accepts((b; a + (a+b)^*; a; b; a), W). \quad (126)$$

We then fold Clause (121) using the definition Clause (126), and we get the following clause:

$$new0([a|W]) \leftarrow new1(W). \quad (127)$$

Now the Clauses (123) and (127) defining the predicate *new0* do not contain any instantiated atom.

Our partial evaluation process continues by considering Clause (126) whose body contains an instantiated atom. By replaying the unfolding steps and the clause deletion steps that lead from Clause (121) to Clauses (124) and (125), and then performing a folding step using Clause (126), we get the following two clauses for the predicate *new1*:

$$new1([a|W]) \leftarrow new1(W) \quad (128)$$

$$new1([b|W]) \leftarrow accepts((a + (a + b)^*; a; b; a), W). \quad (129)$$

Now in order to get clauses with no instantiated atoms in their bodies, we need to transform Clause (129). Similarly to what we have previously done starting from Clause (121), we first apply some unfolding steps and clause deletion steps, and we then perform, when it is possible, some folding steps using definition clauses already introduced. When folding steps are not possible, we apply the loop absorption strategy and we introduce new definition clauses. By doing so we get the following final program:

$$accepts(((a + b)^*; a; b; a), W) \leftarrow new0(W) \quad (130)$$

$$new0([a|W]) \leftarrow new1(W) \quad (127)$$

$$new0([b|W]) \leftarrow new0(W) \quad (123)$$

$$new1([a|W]) \leftarrow new1(W) \quad (128)$$

$$new1([b|W]) \leftarrow new2(W) \quad (131)$$

$$new2([a|W]) \leftarrow new3(W) \quad (132)$$

$$new2([b|W]) \leftarrow new0(W) \quad (133)$$

$$new3([]) \leftarrow \quad (134)$$

$$new3([a|W]) \leftarrow new1(W) \quad (135)$$

$$new3([b|W]) \leftarrow new2(W) \quad (136)$$

where Clause (130) has been added to the program for comparing the evaluations of the initial program and the evaluations of the residual program w.r.t. the same set of goals, each of which is of the form: “*accepts(((a + b)*; a; b; a), w)*”, for some given word *w* in $\{a, b\}^*$.

From the correctness of the transformation rules it follows that, for every ground goal *G* of the form “*accepts(((a + b)*; a; b; a), w)*”, we have that *G* is true in the least Herbrand model of the initial program iff *G* is true in the least Herbrand model of the final program.

Our final program is equal to the one derived by the Logimix partial evaluator [Mogensen and Bondorf 1993], and it defines a deterministic finite automaton corresponding to the given regular expression, by considering the initial state 0 and the final state 3, and by taking any clause of the form *newh([s|W])* $\leftarrow$ *newk(W)* as defining the transition from state *h* to state *k* with label *s*, for *h* and *k* in $\{0, 1, 2, 3\}$ and *s* in $\{a, b\}$.

Similarly to Partial Evaluation, the *Staging Transformation* [Jørring and Scherlis

1986] is a technique whereby the computation of a given function, say $f(x, y)$, is performed in *stages*, that is, $f(x, y)$ is computed as the value of the expression $f_2(f_1(x), y)$, where f_1 and f_2 are suitable auxiliary functions such that:

$$f(x, y) = f_2(f_1(x), y).$$

The idea behind the Staging Transformation is the assumption that the argument x is known *before* y (for instance, x is known at compile-time and y is known at run-time) and that, having computed the value of $f_1(x)$, we can then very efficiently evaluate f_2 .

Staging Transformation can be viewed as a particular generalization from functions to functions by implicit definition. Indeed, one may apply twice that form of generalization strategy for obtaining from the equation $f(x, y) = \text{expr}$ the functions f_1 and f_2 such that $f(x, y) = f_2(f_1(x), y)$ holds.

4.3 Finite Differencing

Finite Differencing [Paige and Koenig 1982] is a program transformation technique similar to the compiling technique called “reduction of the operator strength.” We now illustrate the basic idea of finite differencing by way of an example.

Example 4.3.1 (Finite differencing via generalization: Accumulation over sequences in monoids). Let us consider the sequence $e_0, \dots, e_{N+1}$ for some $N \geq 0$, and let us assume that we want to compute the sum of the elements of that sequence. That sum can be computed as the value of $f(N+1)$, defined by the following recursive equations:

$$f(0) = e_0$$

$$f(n+1) = e_{n+1} + f(n) \quad \text{for } 0 \leq n \leq N.$$

Let us assume that there exists a function δ such that for any n , with $0 \leq n \leq N$, the element e_n of the given sequence is equal to $\delta(e_{n+1}, n)$, and the computation of the function δ is very efficient. Then the application of the finite differencing technique generates the following program for the computation of the function f :

$$f(0) = e_0$$

$$f(n+1) = h(0, e_{n+1}, n) \quad \text{for } n \geq 0$$

$$h(\text{acc}, z, 0) = \text{acc} + z + e_0$$

$$h(\text{acc}, z, n+1) = h(\text{acc} + z, \delta(z, n+1), n) \quad \text{for } n \geq 0,$$

where the argument *acc* plays the role of an accumulator.

More generally, let us consider the following program:

$$f(0) = e(0) \tag{137}$$

$$f(n+1) = g(e(n+1), f(n)) \quad \text{for } n \geq 0, \tag{138}$$

where $g(x, y)$ is an associative operation with neutral element η (thus g determines a *monoid*), and for any $n \geq 0$, $e(n)$ can be efficiently evaluated from the value of $e(n+1)$ by computing $\delta(e(n+1), n)$ for some given function δ , whereas the direct evaluation of $e(n)$ from the value of n is computationally expensive. Thus for $n \geq 0$ we have that $f(n+1) = g(e(n+1), \dots, g(e(1),$

$e(0)) \dots$). The value of $f(n + 1)$ is said to be the *accumulation* of g over the sequence “ $e(0), e(1), \dots, e(n + 1)$ ”.

In these hypotheses finite differencing, as formalized by the schema transformation proposed in Partsch [1990], produces the following tail recursive program, which is a straightforward generalization of the program given previously:

$$f(0) = e(0) \quad (137)$$

$$f(n + 1) = H(\eta, e(n + 1), n) \quad \text{for } n \geq 0 \quad (139)$$

$$H(acc, z, 0) = g(g(acc, z), e(0)) \quad (140)$$

$$H(acc, z, n + 1) = H(g(acc, z), \delta(z, n + 1), n) \quad \text{for } n \geq 0. \quad (141)$$

Thus finite differencing basically works by performing the evaluation of an argument, $e(n + 1)$ in our case, of a recursive function call by using the value, $e(n)$ in our case, of that argument in the previous function call. This fact may in some cases increase program efficiency.

Now we show that the efficiency improvements due to finite differencing can be obtained via suitable generalization and folding steps. The corresponding transformation process can be done in steps as follows.

- (1) We first look for a tail recursive program. By doing so, the computations to be performed from one recursive call to the next, are only the ones related to the evaluation of a substitution, say θ , which computes the new recursive arguments from the old arguments.
- (2) We then generalize suitable subexpressions, so that the substitution θ can be efficiently evaluated.

We start off by considering the initial program made out of the Equations (137) and (138) with the assumptions on g and δ mentioned before. Then we unfold Equation (138) and we get for $n > 0$:

$$f(n + 1) = g(e(n + 1), f(n)) = \{\text{unfolding}\} = g(e(n + 1), g(e(n), f(n - 1))).$$

To derive a tail recursive program, the expression $g(e(n + 1), g(e(n), f(n - 1)))$ should be an instance of the r.h.s. of Equation (138) via the substitution $\{n = n - 1\}$ which relates the recursive calls of the function f .

Thus we first use the associativity property of g for allowing the matching of the subexpression $f(n - 1)$, and we get:

$$f(n + 1) = g(g(e(n + 1), e(n)), f(n - 1)). \quad (142)$$

Then, in order to fold the r.h.s. of Equation (142) using Equation (138), we need to generalize the subexpression $g(e(n + 1), e(n))$, which does *not* match $e(n + 1)$, to a variable, say y , and we define the function:

$$G(y, n) = g(y, f(n)) \quad \text{for } n \geq 0. \quad (143)$$

Thus we get the following program:

$$f(0) = e(0) \quad (137)$$

$$f(n + 1) = \{\text{folding (138) using (143)}\} = G(e(n + 1), n) \quad \text{for } n \geq 0 \quad (144)$$

$$G(y, 0) = g(y, e(0)) \quad (145)$$

$$\begin{aligned} G(y, n + 1) &= \{\text{by 10.7}\} = g(y, f(n + 1)) = \{\text{unfolding}\} \\ &= g(y, g(e(n + 1), f(n))) = \{\text{associativity of } g\} \\ &= g(g(y, e(n + 1)), f(n)) = \{\text{folding using (143)}\} \end{aligned}$$

$$= G(g(y, e(n+1)), n) \quad \text{for } n \geq 0. \quad (146)$$

This program is tail recursive, but unfortunately, it is *not* efficient because by Equation (146) the folding substitution $\{y = g(y, e(n+1)), n+1 = n\}$, which computes the new arguments of G from the old ones, requires the expensive evaluation of $e(n+1)$. We can avoid this drawback by unfolding Equation (146) and performing a generalization step as follows:

$$\begin{aligned} G(y, n+1) &= G(g(y, e(n+1)), n) = \{\text{unfolding using (146)}\} \\ &= G(g(g(y, e(n+1)), e(n)), n-1). \end{aligned}$$

This last expression $G(g(g(y, e(n+1)), e(n)), n-1)$ is an instance of the previous expression $G(g(y, e(n+1)), n)$ according to the substitution $\{y = g(y, e(n+1)), e(n+1) = e(n), n = n-1\}$, which can be efficiently computed if the values of y , $e(n+1)$, and n are known. In particular, given $e(n+1)$ and n we can compute $e(n)$ by using the function δ .

The values of y , $e(n+1)$, and n are known during the recursive evaluation of G if we promote $e(n+1)$ to be an argument of a new function, say H , defined by generalization from functions to functions applied to the r.h.s. $G(g(y, e(n+1)), n)$ of Equation (146). Thus H should satisfy the following:

$$H(y, e(n+1), n) = G(g(y, e(n+1)), n) \quad \text{for } n \geq 0. \quad (147)$$

For the function H we have:

$$\begin{aligned} H(y, e(1), 0) &= \{\text{by (147)}\} = G(g(y, e(1)), 0) = \{\text{by (143) and (137)}\} \\ &= g(g(y, e(1)), e(0)) \end{aligned} \quad (148)$$

$$\begin{aligned} H(y, e(n+2), n+1) &= \{\text{by (147)}\} = G(g(y, e(n+2)), n+1) = \{\text{by (146)}\} \\ &= G(g(g(y, e(n+2)), e(n+1)), n) = \{\text{folding using (147)}\} \\ &= H(g(y, e(n+2)), e(n+1), n) \\ &= \{\text{by } e(n+1) = \delta(e(n+2), n+1)\} \\ &= H(g(y, e(n+2)), \delta(e(n+2), n+1), n) \quad \text{for } n \geq 0. \end{aligned} \quad (149)$$

The definition of the function H is obtained by replacing in Equations (148) and (149) the second argument by a variable, say z . As a result, we get the following final program:

$$f(0) = e(0) \quad (137)$$

$$\begin{aligned} f(n+1) &= G(e(n+1), n) = \{\text{neutrality of } \eta\} = G(g(\eta, e(n+1)), n) \\ &= \{\text{by 147}\} = H(\eta, e(n+1), n) \quad \text{for } n \geq 0 \end{aligned} \quad (150)$$

$$H(y, z, 0) = g(g(y, z), e(0)) \quad (148^*)$$

$$H(y, z, n+1) = H(g(y, z), \delta(z, n+1), n) \quad \text{for } n \geq 0. \quad (149^*)$$

This program is equal to the one made out of Equations (137), (139), (140) and (141) which is produced by the finite differencing technique. Notice that our derivation avoids the insights that are often required by the finite differencing technique, such as the

understanding that the first argument of H is an accumulator variable that holds the value of $g(\dots g(g(\eta, e(n+1)), e(n)), \dots, e(k))$ for some k , with $0 \leq k \leq n+1$, during the computation of $f(n+1)$.

Now, if we want to efficiently compute the following function ss (see Partsch [1990, p. 295]) for computing the sum of squares:

$$ss(0) = 0$$

$$ss(n+1) = (n+1)^2 + ss(n) \quad \text{for } n \geq 0,$$

we may use Equations (137), (150), (148*), and (149*), where we have that $g(x, y) = x + y$; $e(0) = \eta = 0$; $e(n) = n^2$; and $\delta(x, y) = x - 2y - 1$. This expression for the function δ is derived from the fact that, given $x = (n+1)^2$ and $y = n$, the value of n^2 can be efficiently computed by evaluating $x - 2y - 1$. Thus we get the following program that avoids repeated squaring operations:

$$ss(0) = 0$$

$$ss(n+1) = H(0, (n+1)^2, n) \quad \text{for } n \geq 0$$

$$H(y, z, 0) = y + z$$

$$H(y, z, n+1) = H(y+z, z-2(n+1)-1, n) \quad \text{for } n \geq 0.$$

5. RELATED TECHNIQUES FOR PROGRAM DEVELOPMENT AND FINAL DISCUSSION

Due to space limitations we are not able to present here some other program transformation techniques that have been proposed in the literature (see, for instance, Arsac and Kodratoff [1982], Berry [1976], Huet and Lang [1978], Meertens [1987], Pepper [1984, 1987], and Wegbreit [1976]).

In this section we briefly indicate some techniques that have been developed in the fields of theoretical computer science and software engineering and can also be used for assisting the programmer during the program transformation process.

First of all we want to mention the technique for proving program properties called *abstract interpretation* [Abramsky and Hankin 1987; Cousot and Cousot 1977; Debray 1992]. By that technique one may acquire detailed knowledge about various program properties, such as strictness of functions, types of arguments, data flow analysis, destructiveness of updatings, termination of evaluations, groundness of terms, functionality of predicates, and the like, and those properties are then used for driving the program transformation process.

Notice, however, that in general only a “partial knowledge” about those properties is achievable because of undecidability limitations. Notice also that the computation relative to the abstract interpretation analysis should always be terminating. Thus one has to choose the domain of the abstraction with particular care so as to get the maximum amount of information from the minimum amount of computation.

Of particular relevance for the transformation process is the abstract interpretation called “binding-time analysis,” which tells us which expressions should be unfolded when part of the input data is known at compile-time [Launchbury 1991; Mycroft 1981; Nielson and Nielson 1988; Jones et al. 1993]. A general framework for the use of abstract interpretations when transforming logic programs is presented in Boulanger and Bruynooghe [1993]. As an example of the use of abstract interpretations we now present the so-called *Matrix of Lengths method* for the derivation of on-line programs.

Given a program that produces a string as output, this method allows us to derive an equivalent program with *on-line behavior*, that is, a program that produces the output

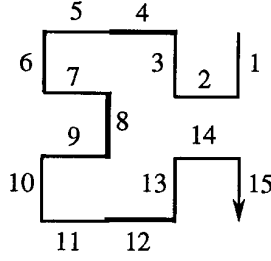


Figure 7 Graphical representation of the string $Hilbert(2) = A(2)$. The moves 1, 2, and 3 are the three elements of $D(1) = S::W::N$.

string incrementally, one element at a time in a given order, and the production of one more output element takes *constant* time and *constant* space after the production of the preceding elements.

Actually, in our example we allow for logarithmic time and logarithmic space w.r.t. the length k of the output string, and we say that a *pseudo on-line* behavior has been achieved. For further details the interested reader may refer to Pettorossi [1987a].

Example 5.1 (The Matrix of Lengths Method: Hilbert Curves). Let us consider the free monoid H^* of sequences of elements, also called moves, generated by the set $H = \{N, S, E, W\}$, where N, S, E , and W denote the elementary moves towards North, South, East, and West, respectively. Let *skip* be the empty move and “ $::$ ” denote the concatenation of moves in H^* . Let us consider the following program for computing the function $Hilbert: N \rightarrow H^*$, that is, the Hilbert Curve of any given order $n \geq 0$ [Wirth 1976]:

$$Hilbert(n) = A(n) \quad (151)$$

$$A(0) = B(0) = C(0) = D(0) = skip \quad (152)$$

$$A(n+1) = D(n)::W::A(n)::S::A(n)::E::B(n) \quad (153)$$

$$B(n+1) = C(n)::N::B(n)::E::B(n)::S::A(n) \quad (154)$$

$$C(n+1) = B(n)::E::C(n)::N::C(n)::W::D(n) \quad (155)$$

$$D(n+1) = A(n)::S::D(n)::W::D(n)::N::C(n), \quad (156)$$

where all equations hold for $n \geq 0$. In Figure 7 we show the string $Hilbert(2)$. In that figure we have represented each move, say the m th one, as a segment assumed to be oriented from the $(m-1)$ st move to the $(m+1)$ st move. For instance, we have that the 10th move (from the left) of $Hilbert(2)$ is S (not N).

Now we show through an example that for any $m > 0$ and $n \geq 0$ we can compute the m th move of $Hilbert(n)$ in logarithmic time, and thus, we derive a pseudo on-line algorithm.

We first derive a linear recursive program for $Hilbert(n)$ by applying the tupling strategy as follows. We tuple together the four function calls $A(n)$, $B(n)$, $C(n)$, and $D(n)$ because they share common subcomputations. Indeed, $A(n)$, $B(n)$, $C(n)$, and $D(n)$ all share $A(n-2)$.

We introduce the function: $Z(n) = \langle A(n), B(n), C(n), D(n) \rangle$, and for $n \geq 0$ we have:

$$Hilbert(n) = \pi_1(Z(n)) \quad (157)$$

$$Z(0) = \langle skip, skip, skip, skip \rangle \quad (158)$$

$$Z(n+1) = \langle d :: W :: a :: S :: a :: E :: b, \quad c :: N :: b :: E :: b :: S :: a, \quad (159)$$

$$b :: E :: c :: N :: c :: W :: d, \quad a :: S :: d :: W :: d :: N :: c \rangle$$

$$\textbf{where } \langle a, b, c, d \rangle = Z(n) \text{ for } n \geq 0.$$

Now we apply the Matrix of Length method as follows. As already mentioned, this method is based on a particular application of the *abstract interpretation* technique in which we consider the lengths of the strings in H^* , instead of the strings themselves.

Thus, with reference to the function $Z(n)$, we consider the lengths of its components and we introduce the function $L(n) = \langle L_A(n), L_B(n), L_C(n), L_D(n) \rangle$, where $L_A(n), \dots, L_D(n)$ are the lengths of the strings $A(n), \dots, D(n)$, respectively.

From Equation (157) we have that the length of $Hilbert(n)$ is $L_A(n)$ for any $n \geq 0$. By performing the abstract interpretation of Equations (158) and (159) we get the following equations defining a linear recurrence relation:

$$L(0) = \langle 0, 0, 0, 0 \rangle \quad (160)$$

$$\begin{aligned} L(n+1) = \langle 2L_A + L_B + L_D + 3, & \quad L_A + 2L_B + L_C + 3, \\ L_B + 2L_C + L_D + 3, & \quad L_A + L_C + 2L_D + 3 \rangle \end{aligned} \quad (161)$$

$$\textbf{where } \langle L_A, L_B, L_C, L_D \rangle = L(n) \text{ for } n \geq 0.$$

Then, in order to find a move of $Hilbert(n)$, we construct the Matrix of Lengths, say M , from column $j = 0$ up to column $j = n$. For $i = A, B, C, D$ and $j = 0, 1, \dots, n$ the entry of M at row i and column j , denoted by $M(i, j)$, is $L_i(j)$. This matrix can be used to efficiently compute a move of $Hilbert(n)$ as we now show through an example.

Suppose that we want to compute the 10th move of $Hilbert(2)$. In this case we have $n = 2$, and by applying Equations (160) and (161) we have the following matrix M :

	$j=0$	$j=1$	$j=2$
$i = A$	0	3	15
$i = B$	0	3	15
$i = C$	0	3	15
$i = D$	0	3	15

The application of our method continues by looking at one column at a time, from column $j = n (=2)$ down to column $j = 0$ until we find, as we now indicate, the desired move. For each column j we compute a triplet of values, say $\langle i, j, k \rangle$, where i is an element of $\{A, B, C, D\}$, denoting the component of $Z(j)$ whose k th move we want to compute.

We start from the triplet $\langle A, 2, 10 \rangle$, because we want to compute the 10th move of $Hilbert(2)$, that is, $A(2)$ (see Equation (151)). Then, for any column $j > 0$ we compute from the triplet $\langle i, j, m \rangle$ for column j the new triplet $\langle newi, j - 1, newm \rangle$ for column $j - 1$, as follows. We first consider the equation defining $i(j)$, that is, $A(2)$ in our case:

$$A(2) = D(1) :: W :: A(1) :: S :: A(1) :: E :: B(1). \quad (162)$$

We then apply to the r.h.s. of this equation the abstract interpretation that considers the length of the strings, and thus, in our case, we associate with $D(1) :: W :: A(1) :: S :: A(1) :: E :: B(1)$ the list $[3, 1, 3, 1, 3, 1, 3]$, because $L_A(1) = L_B(1) = L_D(1) = 3$. From this list we compute the values of $newi$ and $newm$ by considering that the length of the substring $D(1) :: W :: A(1) :: S$ is 8 ($= 3 + 1 + 3 + 1$). Hence, the 10th move of $A(2)$ is a move of $A(1)$; actually, it is the second move (from the left) of $A(1)$ (because $10 - 8 = 2$). Thus we get the triplet relative to column $j = 1$. It is $\langle A, 1, 2 \rangle$ because $A(1)$ gives us the two components A and 1, and the second move gives us the third component 2.

Finally, we compute the triplet relative to column $j = 0$ and we do so by considering the following equation:

$$A(1) = D(0) :: W :: A(0) :: S :: A(0) :: E :: B(0) \quad (163)$$

and the associated list of lengths: $[0, 1, 0, 1, 0, 1, 0]$. Hence, the second move of $A(1)$ is S , because the lengths of $D(0)$, $A(0)$, and $B(0)$ are all 0.

It remains to be proved that for the computations we have previously indicated we only need at most a logarithmic amount of time and space with respect to n .

First of all, let us notice that if the abstract interpretation of the given equations gives rise to linear recurrence relations with constant coefficients, we need only a fixed number of columns of the Matrix of Lengths M for computing one more column. We do not need to keep the entire matrix in memory, and we only need a constant number of memory cells (four, in our case, to store one column).

We also need a logarithmic amount of time to compute the elements of a new column of M from the elements of the stored columns, because linear recurrence relations with constant coefficients can be evaluated in logarithmic time (see Example 2.3.5).

Moreover, if the linear recurrence relations for $L(n)$ have solutions that grow exponentially with the parameter n , then when looking for the m th move of the output string, the number of columns in the matrix M is proportional to $\log(m)$, and thus we only need $O(\log(m))$ time to compute the m th move.

One may apply the Matrix of Lengths method also to the Towers of Hanoi problem presented in Section 2. In that case the computation of the m th move can be performed in constant time (if we assume that the binary digits of the given number m can be computed in constant time) [Pettorossi 1987a].

When transforming programs by the “rules + strategies” approach we cannot change the programming language. This limitation may be overcome by the so-called *program annotation* technique, which has been proposed in Schwarz [1982]. In the annotation approach the improvement of performances is obtained by deriving from the given program written in the language L , the corresponding “annotated program” written in the language L_a , for which we can provide an efficient evaluator. Annotations may refer, for instance, to the use of memo-functions, the call-by-value or call-by-need mode of evaluation, and they can be determined by automatic procedures.

Program transformation techniques are also related to *program synthesis* techniques, in the sense that the initial program can be considered as the initial

specification, and the final program can be considered as the implementation of that specification. We may refer the reader to Manna and Waldinger [1991] and Deville and Lau [1994]. However, it is usually understood that the techniques for program synthesis are much more powerful than the simple manipulations that are allowed within the usual transformational approach. If we allow the use of very sophisticated *laws* or *properties* during program transformation (much more sophisticated than the associativity of *max* used in Example 2.3.4) then it is very difficult to separate the areas of program synthesis and program transformation.

A particular issue for which program transformation is related to program synthesis is the one concerning the *invention of data structures*. Indeed, when deriving programs following the “rule + strategies” approach, often the data structures are *dynamically* invented whereas the transformation process progresses, and no fixed choice is made at the beginning of the transformation process [Wirth 1976]. For instance, in Example 2.3.6 the introduction of the nonhomogeneous array $R(l, y)$ consisting of one function and one list, is determined by the need-for-folding.

There are also techniques by which program synthesis may be viewed as part of program transformation. Among those techniques we would like to mention the so-called *completion techniques* [Dershowitz 1985]. In that paper it is shown how a new program can be derived from an old one by successive applications of semantics-preserving rewriting steps each of which can be viewed as a program transformation step.

Recent developments have indicated a new interesting way of considering the process of program development (or transformation) from specifications. This approach is based on *Constructive Type Theories* (see, for instance, Bates and Constable [1985], Burstall et al. [1991], Coquand and Huet [1988], Galmiche [1991], and Sintzoff et al. [1989]) where a program can be derived from the constructive proof of the existence of an element in a suitable domain type.

The transformational methods for developing programs are also closely related to methods for *program construction* (see, for instance, Deville [1990] and Sterling and Kirschenbaum [1993]), where complex programs are developed by enhancing and composing together simpler programs. However, the basic ideas and objectives of program construction are different from those of program transformation. In particular, the starting point of the methods for program construction is not a complete program, but a possibly incomplete and not fully formalized specification. Moreover, the main objective of program construction is the improvement of the efficiency in software production, rather than the improvement of the efficiency of derived programs.

Other techniques related to the transformation strategies we have described, such as *program slicing*, *program integration*, and *program enhancement*, have been described in Lakhotia and Sterling [1988] and Reps [1990].

The presentation of rules and strategies for program derivation and transformation could have been done using *other formalisms* or frameworks, such as the ones proposed in Backus [1978], Bauer et al. [1985], and Bird and Meertens [1987]. In those formalisms strategies take the form of theorems and equations that are used to rewrite programs. We could have also considered the case of Pascal-like programs [Illsley 1988; Smith 1985], where strategies resemble more closely the ones often used in theorem proving, such as divide-and-conquer, global search, local search, and pruning.

We have considered the transformation rules and strategies in the case of functional languages and then, separately, in the case of logic languages. In the literature, however, there are approaches for the integration of the two language paradigms (for instance, Bosco et al. [1987], Darlington and Guo [1979], van

Emden and Yukawa [1987], Fribourg [1985], Goguen and Meseguer [1986], Lincoln et al. [1994], Reddy [1987], and Subrahmanyam and You [1986]), but no sets of rules have been proposed for transforming programs written using those integrated formalisms.

Some work has been done in the direction of translating logic programs into functional programs [Gardarin et al. 1989; Reddy 1984] and vice versa [Togashi and Noguchi 1987]. These translations may allow improvements of efficiency, because one can exploit the functionality of some predicates and one can also use very efficient techniques for the evaluation of recursive equations. The translation of functional programs into logic programs has the advantage that one can “invert” the computation of functions by deriving the inputs corresponding to given outputs via the relational calculus provided by the logic programming setting.

Concerning the limitation of the transformation approach, the reader should notice that, in general, transformation strategies *cannot* be complete in the sense that their ability of deriving new programs from old programs is limited by the fact that the semantics equivalence of programs is undecidable. Thus there is no algorithm that for any given initial program explores in finite time all equivalent programs that are more efficient than the initial program.

Moreover, the transformation methodology based on the kind of rules we have presented, such as unfolding, folding, and the like, has some intrinsic complexity limitations in the sense that there exists a bound on the efficiency improvements that can be obtained via those rules. This has been shown in Zhu [1994].

A final issue is about the *mechanization* of the transformation rules and strategies for program development. The description of some systems that have been already implemented and are under development can be found in Alexandre et al. [1992], Aerts and Van Besien [1991], Bauer et al. [1987], Boyle [1984], Cai and Paige [1990], Feather [1982; 1987], Krieg and Brückner [1989], Partsch and Steinbrüggen [1983], Smith [1993], and Wile [1982].

ACKNOWLEDGMENTS

We would like to thank our colleagues of the IFIP Working Group 2.1 on *Algorithmic Languages and Calculi*. Their encouragement throughout the years of our involvement in the area of program transformation and derivation is greatly appreciated. Particular thanks to B. Möller, T. Mogensen, and H. Partsch for their suggestions and comments, and to A. Haeberer for his invitation to the “Formal Program Development” Seminar in Rio de Janeiro, Brazil, in 1992. For that occasion we wrote a preliminary version of this survey.

REFERENCES

- ABRAMSKY, S. AND HANKIN, C., EDS. 1987. *Abstract Interpretation of Declarative Languages*. Ellis Horwood, Chichester, UK.
- AERTS, K. AND VAN BESIEEN, D. 1991. Autolap: A system for transforming logic programs. Department of Electronics Rep. University of Rome, Tor Vergata, Rome, Italy.
- ALEXANDRE, F., BSAÏES, K., FINANCE, J. P., AND QUÉRÉ, A. 1992. A system for logic program transformation. In *Proceedings of the International Conference on Logic Programming and Automated Reasoning, LPAR '92, Lecture Notes in Computer Science 624*, Springer Verlag, 445–447.
- ARAVINDAN, C. AND DUNG, P. M. 1993. On the Correctness of unfold/fold transformation of normal and extended logic programs. Tech. Rep., Computer Science Division, Asian Institute of Technology, Bangkok, Thailand.
- ARSAC, J. AND KODRATOFF, Y. 1982. Some techniques for recursion removal from recursive functions. *ACM Trans. Program. Lang. Syst.* 4, 2, 295–322.
- AUBIN, R. 1979. Mechanizing structural induction: Part I and II. *Theor. Comput. Sci.* 9, 3, 329–362.
- BACKUS, J. 1978. Can programming be liberated from the Von Neumann style? *Commun. ACM* 21, 8, 613–641.

- BATES, J. L. AND CONSTABLE, R. L. 1985. Proofs as programs. *ACM Trans. Program. Lang. Syst.* 7, 1, 113–136.
- BAUER, F. L., EHLE, H., HORSCH, A., MÖLLER, B., PARTSCH, H., PAUKNER, O., AND PEPPER, P. 1987. The CIP system group. In *The Munich Project CIP. Vol. II. The Program Transformation System CIP-S. Lecture Notes in Computer Science 292*, Springer Verlag.
- BAUER, F. L., BERGHAMMER, R., BROJ, M., DOSCH, W., GEISELBRECHTINGER, F., GNATZ, R., HANGEL, E., HESSE, W., KRIEG-BRÜCKNER, B., LAUT, A., MATZNER, T., MÖLLER, B., NICKL, F., PARTSCH, H., PEPPER, P., SAMELSON, K., WIRSING, M., AND WÖSSNER, H. 1985. The CIP language group. In *The Munich Project CIP. Vol. I. The Wide Spectrum Language CIP-L. Lecture Notes in Computer Science 183*, Springer Verlag.
- BERRY, G. 1976. Bottom-up computation of recursive programs. *RAIRO Inf. Théor.* 10, 3, 47–82.
- BIRD, R. S. 1984a. Using circular programs to eliminate multiple traversal of data. *Acta Inf.* 21, 239–250.
- BIRD, R. S. 1984b. The promotion and accumulation strategies in transformational programming. *ACM Trans. Program. Lang. Syst.* 6, 4, 487–504.
- BIRD, R. S. 1980. Tabulation techniques for recursive programs. *ACM Comput. Surv.* 12, 4, 403–418.
- BIRD, R. S. AND MEERTENS, L. G. L. T. 1987. Two exercises found in a book on algorithmics. In *TC2 WG 2.1 Working Conference on Program Specification and Transformation*, (Bad Tölz, Germany), North Holland, 451–457.
- BJØRNER, D., ERSHOV, A. P., AND JONES, N. D., EDS. 1988. Partial evaluation and mixed computation. In *Proceedings of the IFIP TC2 Workshop on Partial and Mixed Computation* (Gammel Avernæs, Denmark), North Holland.
- BOITEN, E. A. 1990. Views of formal program development. Catholic University of Nijmegen, Nijmegen, The Netherlands, Ph. D. Thesis.
- BOSCO, P. G., GIOVANETTI, E., LEVI, G., MOISO, C., AND PALAMIDESSI, C. 1987. A complete semantic characterization of K-LEAF, a logic language with partial functions. In *Proceedings of the Fourth Symposium on Logic Programming* (San Francisco, CA), IEEE Computer Society Press, Los Alamitos, CA, 1–27.
- BOSSI, A. AND COCCO, N. 1993. Basic transformation operations for logic programs which preserve computed answer substitutions. *J. Logic Program., Special Issue on Partial Deduction 16*, 47–87.
- BOSSI, A., COCCO, N., AND ETALLE, S. 1992. Transforming normal programs by replacement. In *Proceedings of the Third International Workshop on Meta-Programming in Logic, Meta '92* (Uppsala, Sweden), *Lecture Notes in Computer Science 649*, A. Pettorossi, Ed., Springer Verlag 265–279.
- BOULANGER, D. AND BRUYNNOOGHE, M. 1993. Deriving unfold/fold transformations of logic programs using extended OLDT-based abstract interpretation. *J. Symb. Comput.* 15, 495–521.
- BOYLE, J. M. 1984. Lisp to Fortran—program transformation applied. In *Proceedings of the Workshop on Program Transformation and Programming Environments. Nato ASI Series F, Computer and Systems Sciences 8*, P. Pepper, Ed., Springer Verlag, 291–298.
- BOYER, R. S. AND MOORE, J. S. 1975. Proving theorems about LISP functions. *JACM* 22, 1, 129–144.
- BROUGH, D. R. AND HOGGER, C. J. 1991. Grammar-related transformations of logic programs. *New Gen. Comput.* 9, 1, 115–134.
- BRUYNNOOGHE, M., DE SCHREYE, D., AND KREKELS, B. 1989. Compiling Control. *J. Logic Program.* 6, 1 & 2, 135–162.
- BRZOZOWSKI, J. A. 1964. Derivatives of regular expressions. *JACM* 11, 4, 481–494.
- BURSTALL, R. M. AND DARLINGTON, J. 1977. A transformation system for developing recursive programs. *JACM* 24, 1, 44–67.
- BURSTALL, R. M. AND DARLINGTON, J. 1975. Some transformations for developing recursive programs. In *Proceedings of the International Conference on Reliable Software* (Los Angeles, CA), 465–472.
- BURSTALL, R. M. ET AL. 1991. The Lego project. Computer Science Department, Edinburgh University, Edinburgh, Scotland.
- CAI, J. AND PAIGE, R. 1990. The RAPTS transformation system. Computer Science Department, New York University, New York.
- CHATELIN, P. 1976. Program manipulation: To duplicate is not to complicate. Rep. CNRS Laboratoire d'Informatique. Université de Grenoble, France.
- CHIN, W.-N. 1993. Towards an automated tupling strategy. In *Proceedings of the ACM Symposium on Partial Evaluation and Semantics Based Program Manipulation, PEPM '93*, (Copenhagen, Denmark), ACM Press, New York, 119–132.
- CHIN, W.-N. 1992. Safe fusion of functional expressions. In *Proceedings of ACM SIGPLAN Symposium on Lisp and Functional Programming* (San Francisco, CA), ACM Press, New York, 11–20.

- CHIN, W.-N. 1990. Automatic methods for program transformation. Imperial College of Science, Technology and Medicine, University of London, London, U.K., Ph.D. Thesis.
- COQUAND, T. AND HUET, G. 1988. The calculus of constructions. *Inf. Comput.* 76, 95–120.
- COURCELLE, B. 1990. Recursive applicative program schemes. In *Handbook of Theoretical Computer Science. Vol. B*, J. van Leuven, Ed., Elsevier, 459–492.
- COUSOT, P. AND COUSOT, R. 1977. Abstract interpretation: A unified lattice model for static analysis of programs by construction of approximation of fixpoints. In *Proceedings of the Fourth POPL*, 238–252.
- DARLINGTON, J. 1981. An experimental program transformation system. *Artif. Intell.* 16, 1–46.
- DARLINGTON, J. 1972. A semantic approach to automatic program improvement. Department of Machine Intelligence, Edinburgh University, Edinburgh, U.K., Ph.D. Thesis.
- DARLINGTON, J. AND GUO, Y. 1979. The unification of functional and logic languages, towards constraint functional programming. Tech. Rep., Imperial College, London, U.K.
- DEBRAY, S. K., Ed. 1992. Special Issue of *J. Logic Program. Abstract Interpretation*. 12, 2 & 3.
- DEBRAY, S. K. 1988. Unfold/fold transformations and loop optimization of logic programs. In *Proceedings of SIGPLAN 88, Conference on Programming Language Design and Implementation* (Atlanta, GA), *Sigplan Notices* 23, 7, 297–307.
- DEBRAY, S. K. AND WARREN, D. S. 1988. Automatic mode inference for logic programs. *J. Logic Program.* 5, 207–229.
- DESHOWITZ, N. 1985. Synthesis by completion. In *Proceedings of the Ninth International Joint Conference on Artificial Intelligence 1*, 208–214.
- DEVILLE, Y. 1990. *Logic Programming: Systematic Program Development*. Addison-Wesley, Reading, MA.
- DEVILLE, Y. AND BURNAY, J. 1989. Generalization and Program Schemata. In *Proceedings of NACLP '89*, MIT Press, Cambridge, MA, 409–425.
- DEVILLE, Y. AND LAU, K.-K. 1994. Logic program synthesis. *J. Logic Program.* 19 & 20, 321–350.
- VAN EMDEN, M. AND YUKAWA, K. 1987. Logic programming with equations. *J. Logic Program.* 4, 265–288.
- ERSHOV, A. P. 1982. Mixed computation: Potential applications and problems for study. *Theor. Comput. Sci.* 18, 41–67.
- FEATHER, M. S. 1987. A survey and classification of some program transformation techniques. In *Proceedings of the TC2 IFIP Working Conference on Program Specification and Transformation*, (Bad Tölz, Germany), North Holland, 165–195.
- FEATHER, M. S. 1982. A system for assisting program transformation. *ACM Trans. Program. Lang. Syst.* 4, 1, 1–20.
- FRIBOURG, L. 1985. SLOG: A logic programming language interpreter based on clausal superposition and rewriting. In *Proceedings of the IEEE International Symposium on Logic Programming*, (Boston, MA), IEEE Computer Society Press, Los Alamitos, CA, 172–184.
- FUCHS, N. E. AND FROMHERZ, M. P. J. 1992. Schema-based transformations of logic programs. In *Logic Program Synthesis and Transformation, Proceedings LOPSTR '91* (Manchester, UK), T. Clement and K. K. Lau, Eds., Springer-Verlag, 111–125.
- FUTAMURA, Y. 1971. Partial evaluation of computation process—an approach to a compiler-compiler. *Syst. Comput. Controls* 2, 5, 45–50.
- GALLAGHER, J. P. 1993. Tutorial on specialization of logic programs. In *Proceedings of ACM SIGPLAN Symposium on Partial Evaluation and Semantics Based Program Manipulation, PEPM '93* (Copenhagen, Denmark), ACM Press, New York, 88–98.
- GALMICHE, D. 1991. Proofs and program development in AF2. In *Notes of the IFIP W.G. 2.1 Meeting* (Louvain-la-Neuve, Belgium).
- GARDARIN, G., GUESSARIAN, I., AND DE MAINDREVILLE, C. 1989. Translation of logic programs into functional fixpoint equations. *Theor. Comput. Sci.* 63, 253–274.
- GARDNER, P. A. AND SHEPHERDSON, J. C. 1991. Unfold/fold transformations of logic programs. In *Computational Logic, Essays in Honor of Alan Robinson*, J.-L. Lassez and G. Plotkin, Eds., MIT Press, Cambridge, MA, 565–583.
- GOGUEN J. AND MESEGUER J. 1986. Eqlog: Equality, types, and generic modules for logic programming. In *Logic Programming: Functions, Relations, and Equations*, D. De Groot and G. Lindstrom, Eds., Prentice-Hall, Englewood Cliffs, NJ, 295–363. An earlier version appears in *J. Logic Program.* 1, 2, 179–210.
- HOGGER, C. J. 1981. Derivation of logic programs. *JACM* 28, 2, 372–392.

- HUET, G. AND LANG, B. 1978. Proving and applying program transformation expressed with second-order patterns. *Acta Inf.* 11, 31–55.
- HUGHES, R. J. M. 1982. Super-combinators = a new implementation method for applicative languages. In *Proceedings of the 1982 ACM Symposium on Lisp and Functional Programming* (Pittsburgh, PA), 1–10.
- ILLSLEY, M. 1988. Transforming imperative programs. Computer Science Department, Edinburgh University, Edinburgh, Scotland, Ph.D. Thesis.
- JOHNSSON, T. 1985. Lambda lifting: Transforming programs to recursive equations. In *Functional Programming Languages and Computer Architecture* (Nancy, France), *Lecture Notes in Computer Science* 201, Springer Verlag, 190–203.
- JONES, N. D. 1987. Automatic program specialization: A re-examination from basic principles. In *Proceedings of the IFIP TC2 Working Conference on Partial and Mixed Computation* (Eggerup, Denmark), D. Bjørner and A. P. Ershov, Eds., 225–282.
- JONES, N. D., GOMARD, C. K., AND SESTOFT, P. 1993. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, Englewood Cliffs, NJ.
- JONES, N. AND SØNDERGAARD, H. 1987. A semantics-based framework for the abstract interpretation of Prolog. In *Abstract Interpretation of Declarative Languages*, S. Abramsky and C. Hankin, Eds., Ellis Horwood, Chichester, UK, 123–142.
- JØRRING, U. AND SCHERLIS, W. L. 1986. Compilers and staging transformations. In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Programming Languages* (St. Petersburg Beach, FL), 86–96.
- KANAMORI, T. AND HORIUCHI, K. 1987. Construction of logic programs based on generalized unfold/fold rules. In *Proceedings of the Fourth International Conference on Logic Programming*, MIT Press, Cambridge, MA, 744–768.
- KANAMORI, T. AND MAEJI, M. 1986. Derivation of logic programs from implicit definition. Tech. Rep. TR-178, ICOT, Tokyo.
- KAWAMURA, T. AND KANAMORI, T. 1988. Preservation of stronger equivalence in unfold/fold logic program transformation. In *Proceedings of the International Conference on Fifth Generation Computer Systems* (Tokyo), 413–422.
- KOTT, L. 1982. The McCarthy's induction principle: “Oldy” but “Goody.” *Calcolo* 19, 1, 59–69.
- KOTT, L. 1978. About Transformation System: A Theoretical Study. In *Troisième Colloque International sur la Programmation*, Dunod, Paris, France, 232–247.
- KRIEG-BRÜCKNER, B., ED. 1989. COMPASS, A comprehensive algebraic approach to system specification and development. ESPRIT Basic Research Working Group 3264, Objectives, State of the Art, References. University of Bremen, Germany, Bericht no. 6/89.
- LAKHOTIA, A. AND STERLING, L. 1988. Composing recursive logic programs with clausal join. *New Gen. Comput.* 6, 2 & 3, 211–226.
- LAUNCHBURY, J. 1991. Projection factorisations in partial evaluation. In *Distinguished Dissertations in Computer Science*, Cambridge University Press, New York.
- LINCOLN, P., MARTI-OLIET, N., AND MESEGUER, J. 1994. Specification, transformation, and programming of concurrent systems in rewriting logic. In *Proceedings of the DIMACS Conference on Specification of Parallel Algorithms* (Princeton, NJ), G. Blelloch, K. M. Chandy, and S. Jagannathan, Eds., American Mathematical Society, Providence, RI, 309–339.
- LLOYD, J. W. 1987. *Foundations of Logic Programming*, second edition, Springer Verlag.
- LLOYD, J. W. AND SHEPHERDSON, J. C. 1991. Partial evaluation in logic programming. *J. Logic Program.* 11, 217–242.
- MAHER, M. J. 1993. A transformation system for deductive database modules with perfect model semantics. *Theor. Comput. Sci.* 110, 377–403.
- MANNA, Z. 1974. *Mathematical Theory of Computation*. MacGraw-Hill, New York.
- MANNA, Z. AND WALDINGER, R. 1991. Fundamentals of deductive program synthesis. In *Logic, Algebra, and Computation*. F. L. Bauer, Ed., Springer-Verlag, 41–107.
- MARAKAKIS, E. AND GALLAGHER, J. P. 1994. Schema based top-down design of logic programs using abstract data types. In *Proceedings of LOPSTR '94* (Pisa, Italy), L. Fribourg and F. Turini, Eds., Springer Verlag, 138–153.
- MEERTENS, L. G. L. T., ED. 1987. Program specification and transformation. In *Proceedings of the IFIP TC2 W.G. 2.1 Working Conference on Program Specification and Transformation* (Bad Tölz, Germany), North Holland.
- MOGENSEN, T. AND BONDORF, A. 1993. Logimix: A self-applicable partial evaluator for Prolog. In *Logic*

- Program Synthesis and Transformation, Proceedings LOPSTR '92* (Manchester, UK), K.-K. Lau and T. Clement, Eds., Springer-Verlag, 214–227.
- MÖLLER, B. 1993. Derivation of graph and pointer algorithms. In *Formal Program Development. Lecture Notes in Computer Science 755*, B. Möller, H. Partsch, and S. Schuman, Eds. Springer Verlag, 123–160.
- MÖLLER, B. 1991. Relations as a program development language. In *Proceedings of the IFIP TC2 Working Conference on Constructing Programs from Specifications* (Pacific Grove, CA), North Holland.
- MÖLLER, B., PARTSCH, H., AND SCHUMAN, S., EDS. 1993. *Formal Program Development. Lecture Notes in Computer Science 755*, Springer Verlag.
- MORENO-NAVARRO, J. J. AND RODRIGUEZ-ARTEALEJO, M. 1992. Logic programming with functions and predicates: The language Babel. *J. Logic Program.* 12, 191–223.
- MYCROFT, A. 1981. Abstract interpretation and optimizing transformations for applicative programs. CTS 15-81, University of Edinburgh, Scotland, Ph.D. Thesis.
- NIELSON, F. AND NIELSON, H. R. 1988. Automatic binding time analysis for a typed λ -calculus. *Sci. Comput. Program.* 10, 139–176.
- PAIGE, R. AND KOENIG, S. 1982. Finite differencing of computable expressions. *ACM Trans. Program. Lang. Syst.* 4, 3, 402–454.
- PAIGE, R., REIF, J., AND WACHTER, R., EDS. 1993. Parallel algorithm derivation and program transformation. In *Proceedings of the Workshop at Courant Institute of Mathematical Sciences* (New York), Kluwer Academic Publishers.
- PARTSCH, H. A. 1990. *Specification and Transformation of Programs*. Springer Verlag, New York.
- PARTSCH, H. A. AND STEINBRÜGGEN, R. 1983. Program transformation systems. *ACM Comput. Surv.* 15, 199–236.
- PATERSON, M. S. AND HEWITT, C. E. 1970. Comparative schematology. In *Conference on Concurrent Systems and Parallel Computation Project MAC* (Woods Hole, MA), 119–127.
- PEPPER, P. 1987. A simple calculus for program transformation (inclusive of induction). *Sci. Comput. Program.* 9, 221–262.
- PEPPER, P., ED. 1984. Program transformation and programming environments. *Workshop Report, Nato ASI Series F 8*, Springer Verlag.
- PETTOROSSO, A. 1987a. Derivation of efficient programs for computing sequences of actions. *Theor. Comput. Sci.* 53, 151–167.
- PETTOROSSO, A. 1987b. Derivation of programs which traverse their input data only once. In *Proceedings of the Schools on Programming Methodologies* (Rome, Italy) G. Cioni and A. Salwicki, Eds., Academic Press, 165–184.
- PETTOROSSO, A. 1984. Methodologies for transformations and memoing in applicative languages. Edinburgh University, Edinburgh, Scotland, Ph.D. Thesis.
- PETTOROSSO, A. AND BURSTALL, R. M. 1982. Deriving very efficient algorithms for evaluating linear recurrence relations using the program transformation technique. *Acta Inf.* 18, 181–206.
- PETTOROSSO, A. AND PROIETTI, M. 1994. Transformation of logic programs: Foundations and techniques. *J. Logic Program.* 19, 20, 261–320.
- PETTOROSSO, A. AND PROIETTI, M. 1987. Importing and exporting information in program development. In *Proceedings of the IFIP TC2 Workshop on Partial Evaluation and Mixed Computation* (Gammel Avernæs, Denmark), North Holland, 405–425.
- PETTOROSSO, A. AND SKOWRON, A. 1987. Higher order generalization in program derivation. In *Proceedings of Tapsoft '87* (Pisa, Italy), *Lecture Notes in Computer Science 250*, Springer Verlag, 182–196.
- PETTOROSSO, A. AND SKOWRON, A. 1982. Communicating agents for applicative concurrent programming. In *Proceedings of the International Symposium on Programming* (Turin, Italy) *Lecture Notes in Computer Science 137*, Springer Verlag, 305–322.
- PROIETTI, M. AND PETTOROSSO, A. 1991a. Semantics preserving transformation rules for Prolog. In *Proceedings of the ACM Symposium on Partial Evaluation and Semantics Based Program Manipulation, PEPM '91* (New Haven, CT) *SIGPLAN Notices* 26, 9, 274–284.
- PROIETTI, M. AND PETTOROSSO, A. 1991b. Unfolding-definition-folding, in this order, for avoiding unnecessary variables in logic programs. In *Proceedings of the Third International Symposium on Programming Language Implementation and Logic Programming, PLILP '91* (Passau, Germany), *Lecture Notes in Computer Science 528*, Springer Verlag, 347–358.
- PROIETTI, M. AND PETTOROSSO, A. 1994. Synthesis of programs from unfold/fold proofs. In *Proceedings LOPSTR '93* (Louvain-la-Neuve, Belgium), *Workshops in Computing Series*, Springer Verlag, 141–158.

- PROIETTI, M. AND PETTOROSS, A. 1990. Synthesis of eureka predicates for developing logic programs. In *Proceedings of ESOP '90* (Copenhagen), *Lecture Notes in Computer Science* 432, Springer Verlag, 306–325.
- REDDY, U. S. 1987. Functional logic languages, part I. In *Workshop Proceedings on Graph Rewriting. Lecture Notes in Computer Science* 279, J. H. Fasel and R. M. Keller, Eds., Springer Verlag, 401–425.
- REDDY, U. S. 1984. Transformation of logic programs into functional programs. In *Proceedings of the IEEE International Symposium on Logic Programming*, IEEE Computer Society Press, Los Alamitos, CA, 187–197.
- REPS, T. 1990. Algebraic properties of program integration. In *Proceedings of ESOP '90, Lecture Notes in Computer Science* 432, Springer Verlag, 326–340.
- SANDS, D. 1994. Total correctness and improvement in the transformation of functional programs. Tech. Rep. DIKU, University of Copenhagen, Denmark.
- SATO, T. 1992. An equivalence preserving first order unfold/fold transformation system. *Theor. Comput. Sci.* 105, 57–84.
- SCHERLIS, W. L. 1981. Program improvement by internal specialization. In *Proceedings of the Eighth ACM Symposium on Principles of Programming Languages* (Williamsburgh, VA), 41–49.
- SCHWARZ, J. 1982. Using annotations to make recursive equations behave. *IEEE Trans. Softw. Eng. SE* 8, 1, 21–33.
- SEKI, H. 1993. Unfold/fold transformation of general logic programs for the well-founded semantics. *J. Logic Program., Special Issue on Partial Deduction* 16, 1 & 2, 5–23.
- SEKI, H. 1991. Unfold/fold transformation of stratified programs. *Theor. Comput. Sci.* 86, 107–139.
- SEKI, H. AND FURUKAWA, K. 1987. Notes on transformation techniques for generate and test logic programs. In *Proceedings of the International Symposium on Logic Programming*, (San Francisco), IEEE Press, Piscataway, NJ, 215–223.
- SINTZOFF, M., WEBER, M., DE GROOTE, P., AND CAZIN, J. 1989. Definition 1.1 of the generic development language Deva. Tech. Rep., Unité d'Informatique, Université Catholique de Louvain, Louvain-La-Neuve, Belgium.
- SMITH, D. R. 1993. Automatic Derivation of Algorithms. In *Formal Program Development. Lecture Notes in Computer Science* 755, B. Möller, H. Partsch, and S. Schuman, Eds. Springer Verlag, 324–354.
- SMITH, D. R. 1985. The design of divide and conquer algorithms. *Sci. Comput. Program.* 5, 37–58.
- STERLING, L. AND KIRSCHENBAUM, M. 1993. Applying techniques to skeletons. In *Constructing Logic Programs*, J.-M. Jacquet, Ed., Wiley, New York, Chapter 6, 127–140.
- SUBRAHMANYAM, P. A. AND YOU, J. H. 1986. FUNLOG: A computational model integrating logic programming and functional programming. In *Logic Programming: Functions, Relations, and Equations*, D. De Groot, and G. Lindstrom, Eds., Prentice-Hall, Englewood Cliffs, NJ, 157–198.
- TAKEICHI, M. 1987. Partial parametrization eliminates multiple traversals of data structures. *Acta Inf.* 24, 57–77.
- TAMAKI, H. AND SATO, T. 1984. Unfold/fold transformation of logic programs. In *Proceedings of the Second International Conference on Logic Programming* (Uppsala, Sweden), 127–138.
- TARAU, P. AND BOYER, M. 1994. Elementary logic programs. In *Proceedings PLILP '90*, P. Deransart, and J. Małuszynski, Eds., Springer Verlag, 159–173.
- TOGASHI, A. AND NOGUCHI, S. 1987. A program transformation from equational programs into logic programs. *J. Logic Program* 4, 85–103.
- TURNER, D. A. 1979. A new implementation technique for applicative languages. *Softw. Pract. Exper.* 9, 31–49.
- WADLER, P. L. 1985. Listlessness is better than laziness. Computer Science Department, CMU-CS-85-171, Carnegie Mellon University, Pittsburgh, PA, Ph.D. Thesis.
- WAND, M. 1980. Continuation-based program transformation strategies. *JACM* 27, 1, 164–180.
- WEGBREIT, B. 1976. Goal-directed program transformation. *IEEE Trans. Softw. Eng. SE* 2, 69–79.
- WILE, D. 1982. POPART: Producer of parser and related tools. Tech. Rep. RR-82-21, ISI, 4676 Admiralty Way, Marina del Rey, CA.
- WIRTH, N. 1976. *Algorithms + Data Structures = Programs*. Prentice Hall, Inc. Englewood Cliffs, NJ.
- ZHU, H. 1994. How powerful are folding/unfolding transformations? *J. Funct. Program.* 4, 1, 89–112.

Received April 1994; revised April 1995; accepted November 1995